

Documentación técnica y científica del notebook de réplica PINNs

Ecuación vectorial de Burgers 2D con flujo rotacional

Basado en el paper *Equation Identification for Fluid Flows via Physics-Informed Neural Networks*

Materiales analizados:

- Notebook de Colab: `Replica del paper.ipynb`.
- Paper de referencia: New, Villafañe-Delgado y Shugert, ICML 2024, arXiv:2408.17271.

Audiencia: lector con base matemática, pero con experiencia limitada en programación científica.

Índice

1. Resumen ejecutivo	1
2. Descripción general del notebook	1
2.1. Propósito	1
2.2. Flujo de ejecución	1
2.3. Mapa de celdas principales	2
3. Fundamentación teórica desde el paper	2
3.1. Problema inverso para PDEs	2
3.2. Ecuación de Burgers vectorial en 2D	3
3.3. PINNs: red neuronal más residuo físico	3
3.4. Condición de frontera dura	3
3.5. Por qué Newton aparece en la estimación de α y ν	4
3.6. Baseline guiado por datos	4
3.7. Métricas	4
4. Resultados y figuras integradas del notebook	4
4.1. Instantáneas de la solución	5
4.2. Prueba individual de entrenamiento	6
4.3. Figuras comparativas	7
4.4. Tablas de resultados	9
5. Análisis del código por secciones lógicas	10
5.1. Configuración computacional	10
5.2. Generación de datos con FiPy	11
5.3. Muestreo de datos	13
5.4. Red neuronal con frontera dura	14
5.5. Derivadas automáticas y residuo de la PDE	15
5.6. Estimación de α y ν por mínimos cuadrados	17
5.7. Métricas	17
5.8. Entrenamiento guiado por datos	18
5.9. Entrenamiento PINN inverso	19
5.10. Grilla experimental	21
5.11. Gráficas de resultados	22
6. Conexión directa entre código y teoría	25
7. Notas para un lector con poca programación	26
8. Limitaciones de la réplica del notebook	26
9. Conclusiones	27
A. Salidas textuales relevantes del notebook	27
B. Código completo del notebook	27

Índice de figuras

1.	Instantáneas de $u(x, y, t)$ para $\alpha = 2$ y $\nu = 10^{-4}$ generadas por el notebook. La deformación de las bandas horizontales muestra el desarrollo del flujo rotacional. . . .	5
2.	Evolución de pérdidas en una corrida individual del notebook. La escala vertical es logarítmica.	6
3.	Comparación PINN Newton frente a línea base guiada por datos. En la salida guardada del notebook aparecen puntos en la columna de 2048 datos; las columnas de 8192 y 32768 quedan vacías porque la corrida almacenada es demostrativa.	7
4.	Errores del enfoque data-driven en la salida del notebook. La línea horizontal punteada representa el umbral de 10 % usado como referencia de éxito en el paper.	8
5.	Errores de estimación de parámetros para PINNs, separados por optimizador y tamaño de datos.	8
6.	Error relativo de solución para PINNs en la salida del notebook.	9
7.	Figuras 6–8 generadas por el notebook para el baseline data-driven.	9

Índice de tablas

1.	Estructura funcional del notebook.	2
2.	Resultado de la corrida individual del notebook.	9
3.	Tabla 2 publicada en el paper, usada por el notebook como referencia.	10
4.	Traducción entre fórmulas del paper y bloques del notebook.	25

1 Resumen ejecutivo

Este documento explica, con intención didáctica y técnica, el notebook de Colab que replica parcialmente el estudio de New, Villafañe-Delgado y Shugert sobre identificación de ecuaciones para flujos de fluidos mediante redes neuronales informadas por la física, conocidas como PINNs. La meta matemática del notebook es construir una aproximación neuronal de la solución de la ecuación de Burgers vectorial en dos dimensiones y, al mismo tiempo, estimar los parámetros físicos de convección α y difusión ν .

El flujo general del notebook es el siguiente: primero instala dependencias, fija parámetros de ejecución y construye una malla espacio-temporal; después genera datos de referencia con un solucionador de volúmenes finitos; luego define una red neuronal que respeta automáticamente las condiciones de frontera; finalmente entrena dos enfoques, una línea base guiada por datos y una PINN inversa, y produce tablas y figuras comparables con el paper.

Alcance computacional. Este documento se elaboró a partir del notebook y de las salidas ya guardadas en él. No se re-ejecutó la grilla completa de entrenamiento dentro de este entorno, porque la réplica tipo paper exige numerosos entrenamientos y alto costo computacional. Las conclusiones científicas principales se fundamentan en el paper de referencia; las figuras integradas provienen de las salidas presentes en el notebook.

El paper muestra tres ideas centrales: (i) las PINNs pueden usar pocos datos para estimar parámetros físicos en problemas inversos; (ii) la estimación de los parámetros de la PDE no se comporta igual que el entrenamiento de los millones de pesos de una red neuronal; y (iii) para Burgers 2D, como el residuo es lineal en α y ν cuando la red se mantiene fija, un paso tipo Newton o mínimos cuadrados puede estimar esos parámetros de manera más adecuada que un optimizador genérico de primer orden.

2 Descripción general del notebook

2.1 Propósito

El notebook busca reproducir de forma didáctica un experimento de problema inverso. En un problema directo se conocen los parámetros de la ecuación y se busca la solución; aquí ocurre lo contrario: se observan algunos valores de la solución y se intenta reconstruir tanto la solución completa como los parámetros físicos que la generaron.

La ecuación física estudiada es la ecuación de Burgers 2D vectorial, sin fuente y dependiente del tiempo:

$$\mathbf{u}_t + \alpha(\mathbf{u} \cdot \nabla)\mathbf{u} - \nu\Delta\mathbf{u} = 0, \quad \mathbf{u} = (u, v), \quad (x, y, t) \in [0, 1] \times [0, 1] \times [0, 0.499]. \quad (1)$$

En términos prácticos, el notebook construye una red neuronal que recibe tres números, (x, y, t) , y devuelve dos números, (u, v) . Es decir, la red intenta aprender el campo de velocidad completo del fluido.

2.2 Flujo de ejecución

La ejecución está organizada como una tubería de trabajo:

1. **Instalación e importaciones.** Se cargan librerías de cálculo numérico, visualización, aprendizaje automático y solución de PDEs.
2. **Configuración.** Se define si el notebook se ejecuta en modo corto, demostrativo o tipo paper.

3. **Generación de datos.** FiPy resuelve la PDE de Burgers mediante volúmenes finitos para producir datos de referencia.
4. **Muestreo.** Se escogen puntos etiquetados de la solución, puntos de condición inicial y puntos interiores donde se evalúa la PDE.
5. **Definición de la red.** Se implementa una red ED-MLP con una restricción dura de frontera: la forma de la red garantiza que $u = v = 0$ en el borde espacial.
6. **Residuo PDE.** Se usan derivadas automáticas para calcular u_t , u_x , u_y , u_{xx} , u_{yy} y sus análogos para v .
7. **Entrenamiento.** Se entrenan el método guiado por datos y la PINN inversa.
8. **Evaluación y figuras.** Se calculan errores relativos y se grafican comparaciones con el formato del paper.

2.3 Mapa de celdas principales

Tabla 1: Estructura funcional del notebook.

Celdas	Bloque	Función dentro del proyecto
0–3	Presentación e instalación	Explican el objetivo del notebook e instalan dependencias como FiPy, PyTorch, NumPy, Pandas y Matplotlib.
5–13	Configuración global	Fijan semillas, dispositivo de cómputo, modo de ejecución e hiperparámetros.
15–19	Burgers 2D y FiPy	Definen la condición inicial y generan soluciones de referencia.
21–27	Muestreo, red y residuo PDE	Preparan datos, construyen la red y calculan derivadas automáticas.
31–35	Métricas y entrenamiento	Implementan errores, baseline guiado por datos y PINN inversa.
37–42	Pruebas y grilla	Ejecutan una prueba individual y, si se activa, la grilla experimental.
44–59	Figuras y tablas	Replican la lógica visual de las figuras 2–8 y comparan con la Tabla 2 del paper.
61–64	Empaquetado	Guardan resultados y preparan un archivo comprimido para descargar desde Colab.

3 Fundamentación teórica desde el paper

3.1 Problema inverso para PDEs

El paper parte de una formulación general: existe una función desconocida $u : \Omega \rightarrow \mathbb{R}^m$ que satisface operadores diferenciales N_i dentro del dominio y operadores de frontera B_j sobre la frontera:

$$N_i(u; \phi)(\mathbf{x}) = 0, \quad \mathbf{x} \in \Omega, \quad i \in I_N, \quad (2)$$

$$B_j(u; \phi)(\mathbf{x}) = 0, \quad \mathbf{x} \in \Gamma_j, \quad j \in I_B. \quad (3)$$

Aquí ϕ representa parámetros físicos desconocidos. En el notebook, esos parámetros son

$$\phi = (\alpha, \nu), \quad (4)$$

donde α controla la convección y ν controla la difusión.

El problema inverso consiste en dos tareas simultáneas: reconstruir la solución en puntos no observados e identificar los parámetros físicos que hacen coherente esa solución con la PDE.

3.2 Ecuación de Burgers vectorial en 2D

La ecuación usada en el paper y en el notebook es

$$\mathbf{u}_t + \alpha(\mathbf{u} \cdot \nabla)\mathbf{u} - \nu \Delta \mathbf{u} = 0. \quad (5)$$

Como $\mathbf{u} = (u, v)$, la ecuación realmente representa dos ecuaciones acopladas:

$$u_t + \alpha(uu_x + vv_y) - \nu(u_{xx} + v_{yy}) = 0, \quad (6)$$

$$v_t + \alpha(uv_x + vv_y) - \nu(v_{xx} + v_{yy}) = 0. \quad (7)$$

Las condiciones iniciales son

$$u(x, y, 0) = \sin(6\pi y)x(1 - x), \quad (8)$$

$$v(x, y, 0) = -\sin(6\pi x)y(1 - y). \quad (9)$$

Estas condiciones se anulan en la frontera del cuadrado, pero generan flujo rotacional dentro del dominio. Por eso el ejemplo es más exigente que una ecuación de calor desacoplada.

3.3 PINNs: red neuronal más residuo físico

Una PINN aproxima u por una red neuronal u_θ . La palabra “informada por la física” significa que el entrenamiento no solo compara la red con datos observados, sino que también penaliza que la red viole la PDE. El paper escribe la pérdida como

$$L(\theta; \phi) = L_{PDE}(\theta; \phi) + L_{BC}(\theta; \phi) + L_{Data}(\theta). \quad (10)$$

En el notebook, la pérdida PINN se implementa de forma simplificada como

$$L = L_{PDE} + 10L_{Data} + 10L_{IC}. \quad (11)$$

El factor 10 delante de los términos de datos e condición inicial hace que el entrenamiento no ignore las observaciones mientras intenta reducir el residuo de la PDE.

3.4 Condición de frontera dura

El paper evita entrenar explícitamente la pérdida de frontera usando una forma de red que se anula automáticamente en el borde:

$$u_\theta(x, y, t) = x(1 - x)y(1 - y)NN_u(x, y, t; \theta), \quad (12)$$

$$v_\theta(x, y, t) = x(1 - x)y(1 - y)NN_v(x, y, t; \theta). \quad (13)$$

Esto es una restricción dura: no depende de que el entrenamiento la aprenda. Si $x = 0$, $x = 1$, $y = 0$ o $y = 1$, el factor $x(1 - x)y(1 - y)$ vale cero y por tanto $u_\theta = v_\theta = 0$.

3.5 Por qué Newton aparece en la estimación de α y ν

La parte crucial del paper es notar que, para una red fija, el residuo de Burgers es lineal en α y ν . Si se definen

$$A_u = uu_x + vu_y, \quad D_u = u_{xx} + u_{yy}, \quad (14)$$

$$A_v = uv_x + vv_y, \quad D_v = v_{xx} + v_{yy}, \quad (15)$$

los residuos son

$$r_u = u_t + \alpha A_u - \nu D_u, \quad (16)$$

$$r_v = v_t + \alpha A_v - \nu D_v. \quad (17)$$

Para θ fijo, los valores $u_t, A_u, D_u, v_t, A_v, D_v$ ya están determinados por la red. Entonces estimar α y ν se vuelve un problema de mínimos cuadrados:

$$\min_{\alpha, \nu} \left\| b + M \begin{bmatrix} \alpha \\ \nu \end{bmatrix} \right\|_2^2. \quad (18)$$

Esa es la razón por la cual el notebook implementa una función de estimación por mínimos cuadrados, equivalente a un paso de Newton en esta estructura cuadrática.

3.6 Baseline guiado por datos

El paper también propone una línea base: entrenar una red solo con datos y condiciones iniciales, y al final estimar α y ν usando el residuo PDE. Este enfoque permite comparar si la física durante el entrenamiento realmente ayuda. La conclusión general del paper es que una red puramente guiada por datos puede ajustar algunos valores observados, pero no necesariamente recupera bien los parámetros físicos.

3.7 Métricas

El error relativo de solución usado por el paper es

$$E_{rel}(u, u_\theta; X) = \frac{(\sum_{x \in X} \|u(x) - u_\theta(x)\|_2^2)^{1/2}}{(\sum_{x \in X} \|u(x)\|_2^2)^{1/2}}. \quad (19)$$

Para α se usa error relativo escalar:

$$E_\alpha = \frac{|\hat{\alpha} - \alpha|}{\alpha}. \quad (20)$$

Para ν , como varía por órdenes de magnitud, se usa error logarítmico:

$$E_\nu = \frac{|\log_{10}(\hat{\nu} + \varepsilon) - \log_{10}(\nu + \varepsilon)|}{|\log_{10}(\nu + \varepsilon)|}, \quad \varepsilon = 10^{-8}. \quad (21)$$

4 Resultados y figuras integradas del notebook

4.1 Instantáneas de la solución

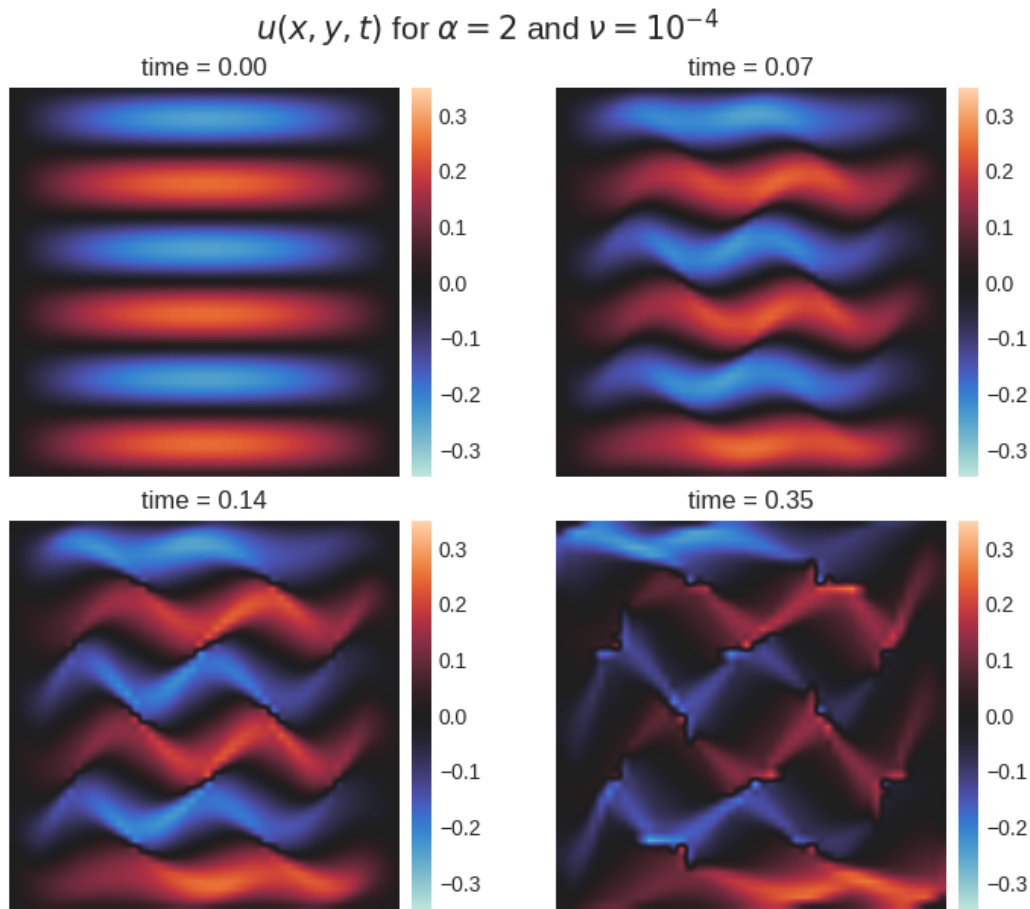


Figura 1: Instantáneas de $u(x, y, t)$ para $\alpha = 2$ y $\nu = 10^{-4}$ generadas por el notebook. La deformación de las bandas horizontales muestra el desarrollo del flujo rotacional.

La figura anterior reproduce la idea visual de la Figura 1 del paper: al inicio las franjas dependen principalmente de y ; luego la convección deforma el campo y aparecen estructuras diagonales y rotacionales.

4.2 Prueba individual de entrenamiento

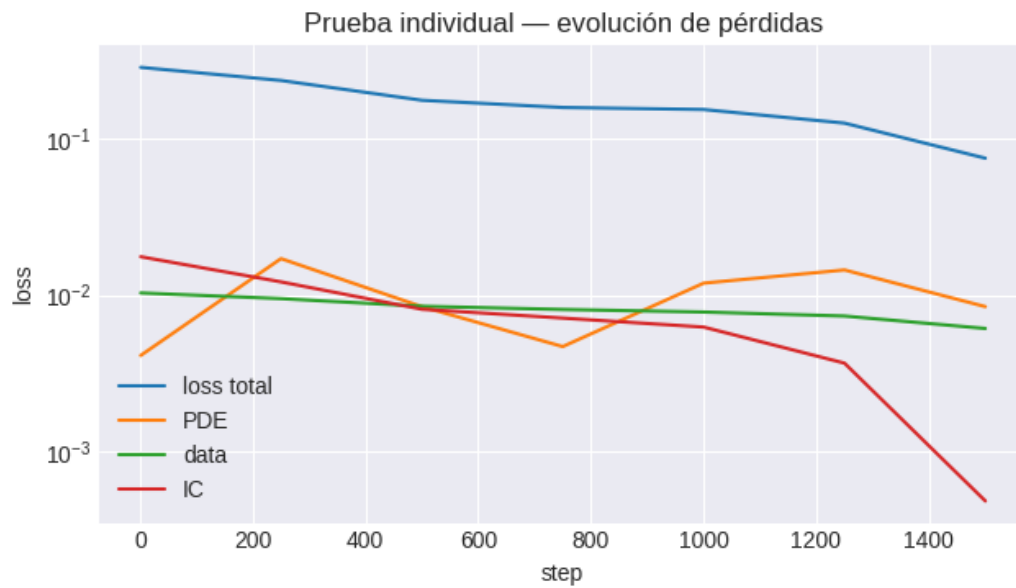


Figura 2: Evolución de pérdidas en una corrida individual del notebook. La escala vertical es logarítmica.

En esta corrida de prueba se usa un entrenamiento reducido. Por eso sus errores no deben interpretarse como réplica final del paper; sirven para comprobar que el pipeline conecta correctamente: solver, muestreo, red, residuo PDE, optimización y métricas.

4.3 Figuras comparativas

Figura 2 — PINN Newton vs data-driven

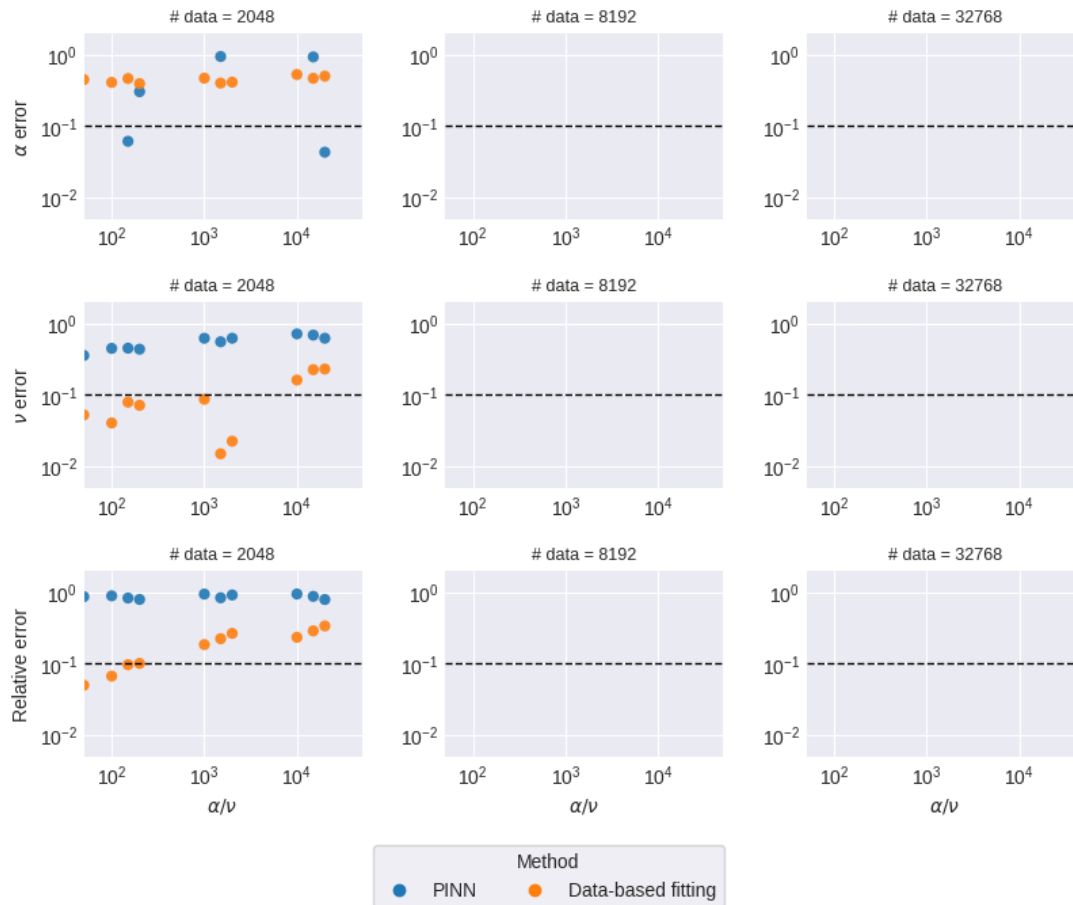


Figura 3: Comparación PINN Newton frente a línea base guiada por datos. En la salida guardada del notebook aparecen puntos en la columna de 2048 datos; las columnas de 8192 y 32768 quedan vacías porque la corrida almacenada es demostrativa.

Figure 3 — Data-driven estimation

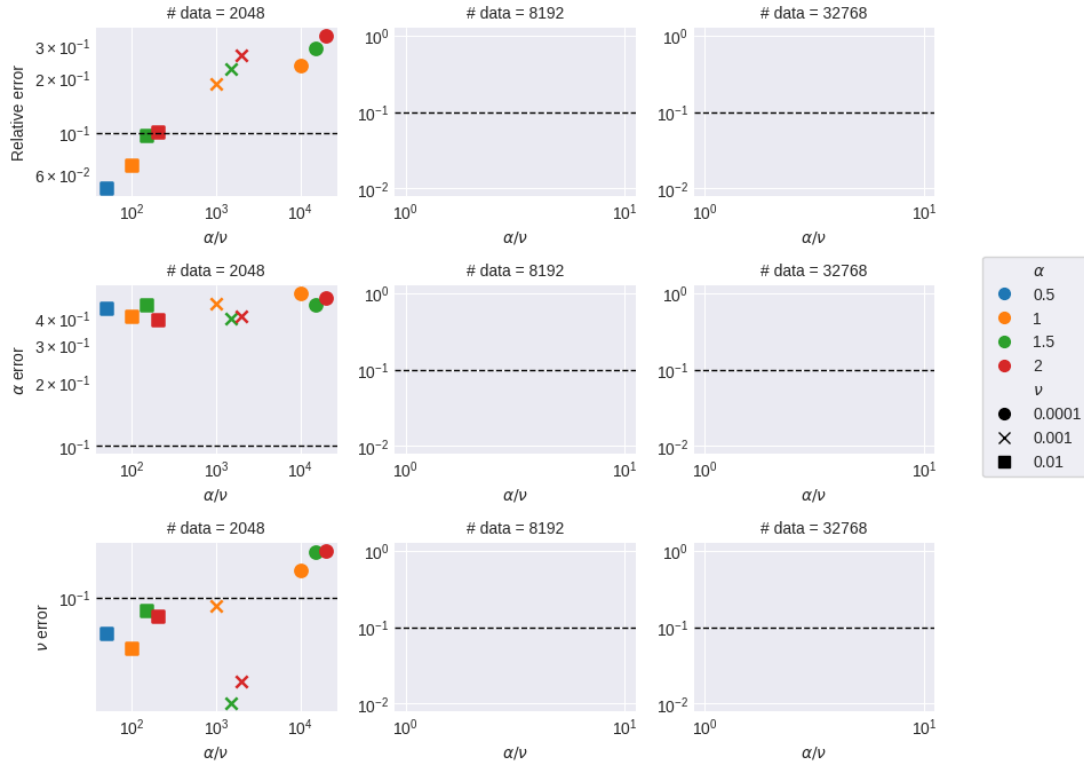


Figura 4: Errores del enfoque data-driven en la salida del notebook. La línea horizontal punteada representa el umbral de 10 % usado como referencia de éxito en el paper.

Figure 4 — PINN parameter estimation errors

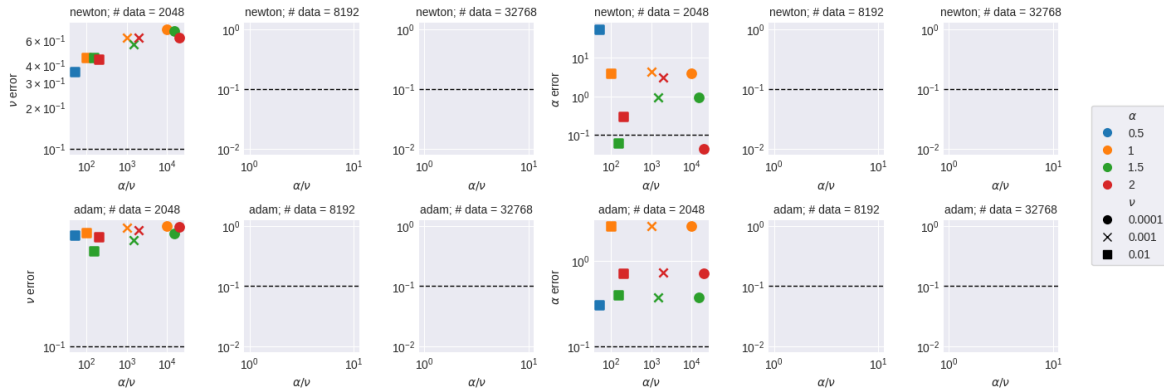


Figura 5: Errores de estimación de parámetros para PINNs, separados por optimizador y tamaño de datos.

Figure 5 — PINN solution relative error

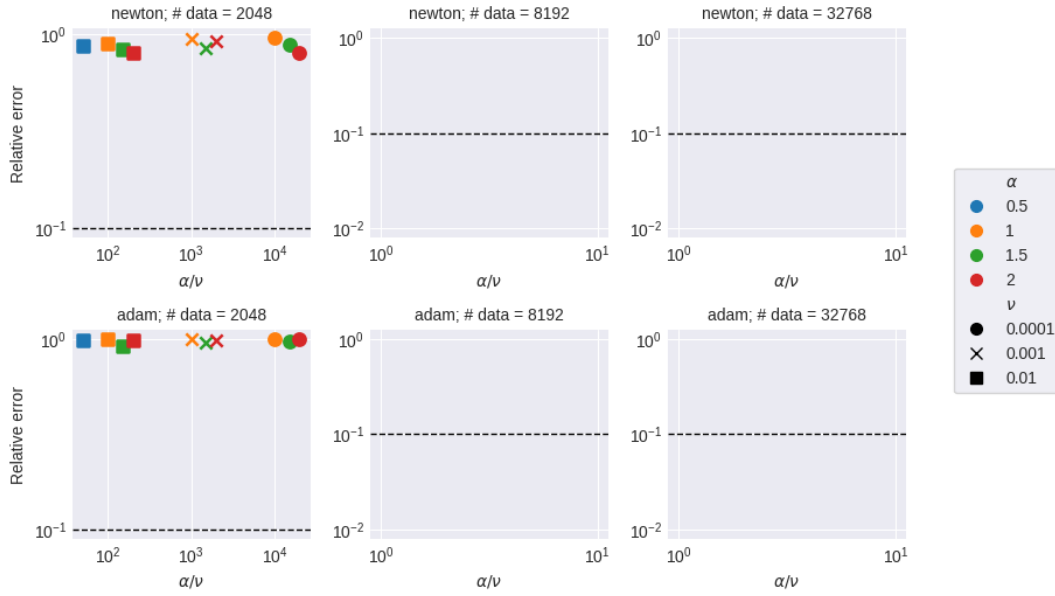


Figura 6: Error relativo de solución para PINNs en la salida del notebook.

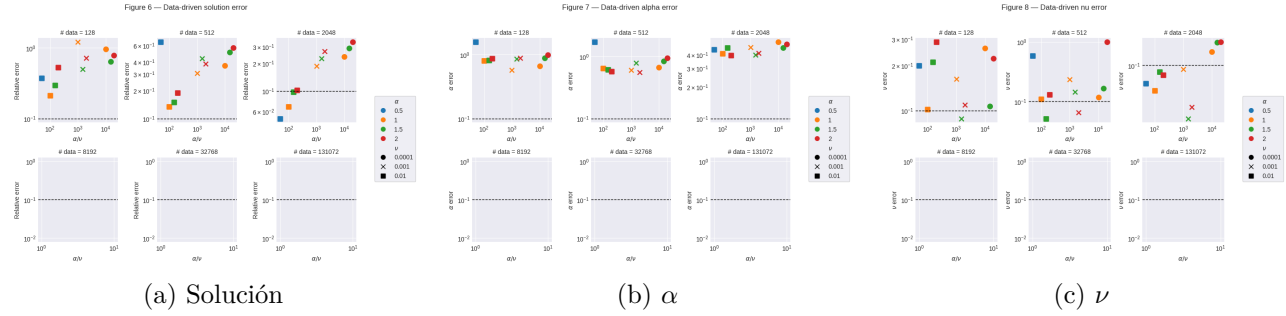


Figura 7: Figuras 6–8 generadas por el notebook para el baseline data-driven.

4.4 Tablas de resultados

Tabla 2: Resultado de la corrida individual del notebook.

Magnitud	Valor real	Estimación / error
α	2.0000	$\hat{\alpha} = 1.1372$, error 0.4314
ν	10^{-4}	$\hat{\nu} = 0.02854$, error log 0.6139
Error relativo de solución	—	0.7798
Datos usados	—	256

Tabla 3: Tabla 2 publicada en el paper, usada por el notebook como referencia.

Optimizador PDE	# datos	Error α	Error relativo	Error ν
Adam	2048	0.080	0.152	0.093
Newton	2048	0.086	0.150	0.056
Adam	8192	0.085	0.139	0.097
Newton	8192	0.085	0.130	0.065
Adam	32768	0.067	0.139	0.101
Newton	32768	0.088	0.126	0.071

La tabla publicada muestra que Newton mejora el error de ν y el error relativo promedio de solución, aunque Adam logra mejor promedio para α en algunos tamaños. Esto coincide con la motivación matemática: Newton aprovecha que, con la red fija, el problema de α y ν tiene estructura de mínimos cuadrados.

5 Análisis del código por secciones lógicas

5.1 Configuración computacional

```

1  # ===== Celda 13 del notebook =====
2  @dataclass
3  class RunConfig:
4      # Datos de referencia
5      nx: int
6      ny: int
7      nt: int
8      t_final: float
9      nonlinear_sweeps: int
10
11     # Arquitectura
12     width: int
13     depth: int
14
15     # Entrenamiento data-driven
16     dd_steps: int
17     dd_batch_data: int
18     dd_batch_ic: int
19     dd_batch_pde: int
20     dd_lr0: float
21     dd_lr_min: float
22
23     # Entrenamiento PINN
24     pinn_steps: int
25     pinn_batch_ic: int
26     pinn_batch_pde: int
27     pinn_lr0: float
28     pinn_lr_min: float
29
30     # Evaluacion y logging
31     eval_points: int
32     print_every: int
33
34
35 def get_config(run_mode: str) -> RunConfig:
36     if run_mode == 'paper':
37         return RunConfig(
38             nx=256, ny=256, nt=72, t_final=0.499, nonlinear_sweeps=4,
39             width=256, depth=10,
40             dd_steps=50000, dd_batch_data=2048, dd_batch_ic=2048, dd_batch_pde=8192,
41             dd_lr0=5e-3, dd_lr_min=1e-6,

```

```

42         pinn_steps=100000, pinn_batch_ic=1024, pinn_batch_pde=2048,
43         pinn_lr0=5e-3, pinn_lr_min=1e-5,
44         eval_points=200000, print_every=5000,
45     )
46     if run_mode == 'demo':
47         return RunConfig(
48             nx=48, ny=48, nt=24, t_final=0.499, nonlinear_sweeps=2,
49             width=80, depth=4,
50             dd_steps=1500, dd_batch_data=512, dd_batch_ic=256, dd_batch_pde=512,
51             dd_lr0=3e-3, dd_lr_min=1e-5,
52             pinn_steps=1500, pinn_batch_ic=256, pinn_batch_pde=512,
53             pinn_lr0=3e-3, pinn_lr_min=1e-5,
54             eval_points=6500, print_every=250,
55         )
56     if run_mode == 'smoke':
57         return RunConfig(
58             nx=48, ny=48, nt=16, t_final=0.499, nonlinear_sweeps=1,
59             width=48, depth=3,
60             dd_steps=80, dd_batch_data=128, dd_batch_ic=128, dd_batch_pde=256,
61             dd_lr0=2e-3, dd_lr_min=1e-5,
62             pinn_steps=100, pinn_batch_ic=128, pinn_batch_pde=256,
63             pinn_lr0=2e-3, pinn_lr_min=1e-5,
64             eval_points=5000, print_every=25,
65         )
66     raise ValueError("RUN_MODE debe ser 'smoke', 'demo' o 'paper'.")
67
68 CFG = get_config(RUN_MODE)
69 CFG

```

Listing 1: Configuración del experimento y clase RunConfig.

Explicación línea por línea de la lógica

- **Dataclass.** La instrucción `@dataclass` crea una clase cuyo propósito principal es guardar parámetros. Una clase es una plantilla para agrupar datos y funciones; aquí se usa solo como contenedor ordenado de valores.
- **Parámetros de datos.** `nx`, `ny`, `nt` y `t_final` determinan el tamaño de la malla de referencia. En modo paper deberían acercarse a $256 \times 256 \times 72$; en modo demo se reducen para poder ejecutar más rápido.
- **Parámetros de red.** `width` es el número de neuronas por capa oculta y `depth` es el número de capas. Más anchura y profundidad aumentan capacidad, pero también costo.
- **Parámetros de entrenamiento.** `dd_steps`, `pinn_steps`, `batch_*` y `lr_*` controlan cuántos pasos se dan, cuántos puntos se usan por actualización y qué tan grande es la tasa de aprendizaje.
- **Selección por modo.** El bloque condicional asigna valores distintos según `RUN_MODE`. Esta estructura evita editar muchos lugares del código para pasar de una prueba corta a una réplica pesada.

5.2 Generación de datos con FiPy

```

1 # ===== Celda 17 del notebook =====
2 def _solution_filename(alpha, nu, cfg: RunConfig):
3     tag = f"alpha:{alpha:g}_nu:{nu:.0e}_nx_{cfg.nx}_ny_{cfg.ny}_nt_{cfg.nt}_tf_{cfg.t_final}"
4     return CACHE_DIR / (tag.replace('.', 'p') + '.npz')
5
6
7 def _cell_values_to_image(phi, nx, ny):

```

```

8     """Convierte valores de celdas FiPy a imagen [ny, nx] para imshow."""
9     arr = np.asarray(phi.value, dtype=np.float64)
10    return arr.reshape((nx, ny), order='F').T
11
12
13    def solve_burgers2d_fipy(alpha=2.0, nu=1e-4, cfg: RunConfig = CFG,
14                             force=False, verbose=True):
15        """
16        Genera una solucion de referencia para Burgers 2D vectorial usando FiPy.
17
18        Ecuaciones:
19            u_t + alpha * div( u * [u, v] ) = nu * Delta u
20            v_t + alpha * div( v * [u, v] ) = nu * Delta v
21
22        Esta es la forma conservativa usada para aproximar el termino no lineal.
23        """
24        path = _solution_filename(alpha, nu, cfg)
25        if path.exists() and not force:
26            data = np.load(path)
27            return {
28                'x': data['x'], 'y': data['y'], 't': data['t'],
29                'U': data['U'], 'V': data['V'],
30                'alpha': float(data['alpha']), 'nu': float(data['nu'])
31            }
32
33        from fipy import CellVariable, FaceVariable, Grid2D
34        from fipy import TransientTerm, DiffusionTerm, PowerLawConvectionTerm
35
36        try:
37            # Si pyAMG esta disponible, FiPy puede usarlo. Si no, cae al solver por defecto.
38            from fipy.solvers.pyAMG import LinearGMRESSolver
39            solver = LinearGMRESSolver(tolerance=1e-10, iterations=2000)
40        except Exception:
41            from fipy import LinearLUSolver
42            solver = LinearLUSolver(tolerance=1e-10, iterations=2000)
43
44        nx, ny, nt = cfg.nx, cfg.ny, cfg.nt
45        dx = 1.0 / nx
46        dy = 1.0 / ny
47        dt = cfg.t_final / (nt - 1)
48
49        mesh = Grid2D(dx=dx, dy=dy, nx=nx, ny=ny)
50        x_cc, y_cc = mesh.cellCenters
51        x_val = np.asarray(x_cc)
52        y_val = np.asarray(y_cc)
53
54        u = CellVariable(mesh=mesh, name='u', value=initial_u_np(x_val, y_val), hasOld=True)
55        v = CellVariable(mesh=mesh, name='v', value=initial_v_np(x_val, y_val), hasOld=True)
56
57        # Condicion de frontera homogenea en todo el borde exterior.
58        u.constrain(0.0, where=mesh.exteriorFaces)
59        v.constrain(0.0, where=mesh.exteriorFaces)
60
61        velocity = FaceVariable(mesh=mesh, rank=1)
62
63        eq_u = TransientTerm(var=u) + PowerLawConvectionTerm(coeff=velocity, var=u) ==
64              DiffusionTerm(coeff=nu, var=u)
65        eq_v = TransientTerm(var=v) + PowerLawConvectionTerm(coeff=velocity, var=v) ==
66              DiffusionTerm(coeff=nu, var=v)
67
68        x_grid = (np.arange(nx) + 0.5) * dx
69        y_grid = (np.arange(ny) + 0.5) * dy
70        t_grid = np.linspace(0.0, cfg.t_final, nt)
71
72        U = np.zeros((nt, ny, nx), dtype=np.float64)
73        V = np.zeros((nt, ny, nx), dtype=np.float64)
74        U[0] = _cell_values_to_image(u, nx, ny)
75        V[0] = _cell_values_to_image(v, nx, ny)

```



```

75     iterator = range(1, nt)
76     if verbose:
77         iterator = tqdm(iterator, desc=f'FiPy alpha={alpha}, nu={nu:g}')
78
79     for k in iterator:
80         u.updateOld()
81         v.updateOld()
82
83         for _ in range(cfg.nonlinear_sweeps):
84             # Velocidad convectiva de cara: alpha * (u, v).
85             velocity[:] = (alpha * u.arithmeticFaceValue, alpha * v.arithmeticFaceValue)
86             eq_u.sweep(var=u, dt=dt, solver=solver)
87             eq_v.sweep(var=v, dt=dt, solver=solver)
88
89             U[k] = _cell_values_to_image(u, nx, ny)
90             V[k] = _cell_values_to_image(v, nx, ny)
91
92     np.savez_compressed(path, x=x_grid, y=y_grid, t=t_grid, U=U, V=V,
93                        alpha=np.array(alpha), nu=np.array(nu))
94     return {'x': x_grid, 'y': y_grid, 't': t_grid, 'U': U, 'V': V,
95            'alpha': alpha, 'nu': nu}

```

Listing 2: Generador de datos de referencia con FiPy.

Lectura didáctica

Este bloque convierte la PDE continua en datos numéricos. FiPy es una librería para resolver ecuaciones diferenciales parciales mediante volúmenes finitos. Un volumen finito divide el dominio en pequeñas celdas y conserva flujos entre ellas, una idea muy natural para fluidos.

- **Nombre de archivo.** La función `_solution_filename` crea un nombre único según α , ν , tamaño de malla y tiempo final. Así se evita recalcular soluciones que ya están guardadas.
- **Conversión a imagen.** `_cell_values_to_image` reorganiza los valores internos de FiPy para poder graficarlos con `imshow`.
- **Malla.** La función principal crea una malla uniforme en $[0, 1] \times [0, 1]$.
- **Condición inicial.** Se asignan los valores de $u(x, y, 0)$ y $v(x, y, 0)$ definidos por el paper.
- **Iteración temporal.** Para cada tiempo se actualizan los campos. El notebook usa un número de barridos no lineales configurable para aproximar el término convectivo.
- **Cache.** Al final se guarda todo en un archivo `.npz`, que es un contenedor comprimido de arreglos de NumPy.

5.3 Muestreo de datos

```

1  # ===== Celda 21 del notebook =====
2  def sample_solution_points(solution, n_points, rng=None):
3      """Muestra puntos etiquetados de la solución completa."""
4      if rng is None:
5          rng = np.random.default_rng(SEED)
6
7      x, y, t = solution['x'], solution['y'], solution['t']
8      U, V = solution['U'], solution['V']
9      nt, ny, nx = U.shape
10     total = nt * ny * nx
11     replace = n_points > total
12     idx = rng.choice(total, size=n_points, replace=replace)

```

```

13
14     it = idx // (ny * nx)
15     rem = idx % (ny * nx)
16     iy = rem // nx
17     ix = rem % nx
18
19     X = np.stack([x[ix], y[iy], t[it]], axis=1)
20     Y = np.stack([U[it, iy, ix], V[it, iy, ix]], axis=1)
21     return X.astype(np.float64), Y.astype(np.float64)
22
23
24 def sample_initial_points(n_points, rng=None):
25     """Muestra puntos de condicion inicial t=0."""
26     if rng is None:
27         rng = np.random.default_rng(SEED)
28     x = rng.random(n_points)
29     y = rng.random(n_points)
30     t = np.zeros(n_points)
31     U0 = initial_u_np(x, y)
32     V0 = initial_v_np(x, y)
33     X = np.stack([x, y, t], axis=1)
34     Y = np.stack([U0, V0], axis=1)
35     return X.astype(np.float64), Y.astype(np.float64)
36
37
38 def sample_pde_points(n_points, t_final=0.499, rng=None):
39     """Puntos interiores para el residuo fisico."""
40     if rng is None:
41         rng = np.random.default_rng(SEED)
42     x = rng.random(n_points)
43     y = rng.random(n_points)
44     t = t_final * rng.random(n_points)
45     X = np.stack([x, y, t], axis=1)
46     return X.astype(np.float64)
47
48
49 def to_tensor(a, requires_grad=False):
50     z = torch.tensor(a, dtype=torch.float64, device=DEVICE)
51     return z.requires_grad_(requires_grad)

```

Listing 3: Funciones de muestreo de puntos etiquetados, condición inicial y puntos PDE.

El muestreo es esencial porque el paper estudia qué ocurre cuando se entrena con muy pocos puntos comparados con la solución completa. En programación, un arreglo es una tabla multidimensional de números; en este caso, los arreglos guardan coordenadas (x, y, t) y valores (u, v) .

- `sample_solution_points` selecciona puntos observados de la solución completa. Estos puntos forman la pérdida de datos.
- `sample_initial_points` genera puntos con $t = 0$ para imponer la condición inicial.
- `sample_pde_points` genera puntos interiores donde se evalúa si la red respeta la ecuación diferencial.
- `to_tensor` transforma arreglos de NumPy en tensores de PyTorch. Un tensor es como un arreglo numérico, pero preparado para GPU y derivadas automáticas.

5.4 Red neuronal con frontera dura

```

1 # ===== Celda 23 del notebook =====
2 class EDMLP(nn.Module):
3     """
4     Encoder-Decoder MLP tipo modified MLP.

```

```

5
6 Esta version usa compuertas suaves entre dos codificaciones U y V.
7 Mantiene el espiritu de la ED-MLP usada en PINNs modernos y es suficiente
8 para reproducir la estructura experimental del paper de forma didactica.
9 """
10 def __init__(self, in_dim=3, out_dim=2, width=256, depth=10):
11     super().__init__()
12     self.U = nn.Linear(in_dim, width)
13     self.V = nn.Linear(in_dim, width)
14     self.first = nn.Linear(in_dim, width)
15     self.hidden = nn.ModuleList([nn.Linear(width, width) for _ in range(depth - 1)])
16     self.last = nn.Linear(width, out_dim)
17     self.act = nn.Tanh()
18     self.reset_parameters()
19
20 def reset_parameters(self):
21     for m in self.modules():
22         if isinstance(m, nn.Linear):
23             nn.init.xavier_normal_(m.weight)
24             nn.init.zeros_(m.bias)
25
26 def forward(self, X):
27     U = self.act(self.U(X))
28     V = self.act(self.V(X))
29     H = self.act(self.first(X))
30     for layer in self.hidden:
31         Z = self.act(layer(H))
32         H = (1.0 - Z) * U + Z * V
33     return self.last(H)
34
35
36 class HardBoundaryPINN(nn.Module):
37     """PINN que satisface u=v=0 en la frontera espacial por construccion."""
38     def __init__(self, width=256, depth=10):
39         super().__init__()
40         self.net = EDMLP(in_dim=3, out_dim=2, width=width, depth=depth)
41
42     def forward(self, X):
43         raw = self.net(X)
44         x = X[:, 0:1]
45         y = X[:, 1:2]
46         factor = x * (1.0 - x) * y * (1.0 - y)
47         return factor * raw

```

Listing 4: Arquitectura ED-MLP y PINN con condición de frontera exacta.

Una red neuronal es una composición de funciones. En este caso, la entrada es (x, y, t) y la salida cruda es (NN_u, NN_v) . La clase `HardBoundaryPINN` multiplica esa salida por $x(1-x)y(1-y)$, lo cual impone exactamente las condiciones de frontera homogéneas.

- **EDMLP.** La arquitectura codificador-decodificador mezcla transformaciones internas para aumentar la expresividad de la red.
- **Función de activación.** `tanh` es una función suave. Esta suavidad importa porque la PDE requiere derivadas de primer y segundo orden.
- **Módulos de PyTorch.** Una clase que hereda de `nn.Module` puede guardar capas y participar en el entrenamiento automático de PyTorch.
- **Frontera dura.** La red no necesita aprender $u = v = 0$ en la frontera; la forma algebraica lo garantiza.

5.5 Derivadas automáticas y residuo de la PDE

```

1  # ===== Celda 25 del notebook =====
2  def grad(outputs, inputs):
3      return torch.autograd.grad(
4          outputs, inputs,
5          grad_outputs=torch.ones_like(outputs),
6          create_graph=True,
7          retain_graph=True,
8          only_inputs=True,
9      )[0]
10
11
12  def pde_piezas(model, X):
13      """
14      Devuelve las piezas del residuo:
15      u_t, v_t, A_u, A_v, D_u, D_v.
16      """
17      X = X.clone().detach().requires_grad_(True)
18      pred = model(X)
19      u = pred[:, 0:1]
20      v = pred[:, 1:2]
21
22      gu = grad(u, X)
23      gv = grad(v, X)
24
25      u_x, u_y, u_t = gu[:, 0:1], gu[:, 1:2], gu[:, 2:3]
26      v_x, v_y, v_t = gv[:, 0:1], gv[:, 1:2], gv[:, 2:3]
27
28      u_xx = grad(u_x, X)[:, 0:1]
29      u_yy = grad(u_y, X)[:, 1:2]
30      v_xx = grad(v_x, X)[:, 0:1]
31      v_yy = grad(v_y, X)[:, 1:2]
32
33      A_u = u * u_x + v * u_y
34      A_v = u * v_x + v * v_y
35      D_u = u_xx + u_yy
36      D_v = v_xx + v_yy
37
38      return u_t, v_t, A_u, A_v, D_u, D_v
39
40
41  def pde_residual(model, X, alpha, nu):
42      u_t, v_t, A_u, A_v, D_u, D_v = pde_piezas(model, X)
43      r_u = u_t + alpha * A_u - nu * D_u
44      r_v = v_t + alpha * A_v - nu * D_v
45      return r_u, r_v
46
47
48  def pde_loss(model, X, alpha, nu):
49      r_u, r_v = pde_residual(model, X, alpha, nu)
50      return (r_u**2).mean() + (r_v**2).mean()

```

Listing 5: Cálculo de derivadas y piezas del residuo de Burgers.

La derivada automática calcula derivadas de una función representada por código. No usa diferencias finitas explícitas; PyTorch construye un grafo de operaciones y aplica la regla de la cadena.

- `grad` envuelve `torch.autograd.grad`. Su papel es obtener derivadas de las salidas respecto a las entradas.
- `pde_piezas` calcula u_t , u_x , u_y , v_t , v_x , v_y y derivadas segundas.
- Las cantidades A_u , A_v representan convección; D_u , D_v representan difusión.
- `pde_loss` calcula el promedio cuadrático de r_u y r_v . Si esta pérdida es pequeña, la red se comporta como una solución aproximada de la PDE.

5.6 Estimación de α y ν por mínimos cuadrados

```

1 # ===== Celda 27 del notebook =====
2 @torch.no_grad()
3 def _safe_scalar(x):
4     if isinstance(x, torch.Tensor):
5         return float(x.detach().cpu().item())
6     return float(x)
7
8
9 def estimate_alpha_nu_least_squares(model, X_pde, ridge=1e-12, clip_nu=True):
10     """Estimacion cerrada de alpha y nu para red fija."""
11     u_t, v_t, A_u, A_v, D_u, D_v = pde_piezas(model, X_pde)
12
13     b = torch.cat([u_t, v_t], dim=0)                # [2N, 1]
14     col_alpha = torch.cat([A_u, A_v], dim=0)        # [2N, 1]
15     col_nu = torch.cat([-D_u, -D_v], dim=0)         # [2N, 1]
16     M = torch.cat([col_alpha, col_nu], dim=1)        # [2N, 2]
17
18     # Min || b + M phi ||^2 => (M^T M) phi = - M^T b.
19     A = M.T @ M + ridge * torch.eye(2, dtype=torch.float64, device=DEVICE)
20     rhs = -M.T @ b
21     phi = torch.linalg.solve(A, rhs).flatten()
22
23     alpha_hat = phi[0]
24     nu_hat = phi[1]
25     if clip_nu:
26         nu_hat = torch.clamp(nu_hat, min=0.0)
27
28     return alpha_hat.detach(), nu_hat.detach()

```

Listing 6: Estimación cerrada de α y ν para red fija.

Este es uno de los bloques más importantes del notebook. En vez de ajustar α y ν con pequeños pasos de Adam, construye un sistema lineal a partir del residuo. La solución de mínimos cuadrados busca los valores que reducen globalmente el residuo cuadrático en los puntos PDE muestreados.

- `@torch.no_grad()` en `_safe_scalar` indica que no se necesita guardar historia para derivadas.
- `estimate_alpha_nu_least_squares` llama a `pde_piezas`, construye la matriz M y el vector b .
- `torch.linalg.solve` resuelve las ecuaciones normales regularizadas.
- `clip_nu=True` impide que la viscosidad estimada sea negativa, lo cual no tendría interpretación física en este contexto.

5.7 Métricas

```

1 # ===== Celda 31 del notebook =====
2 def relative_solution_error(model, solution, n_eval=None, rng=None):
3     if rng is None:
4         rng = np.random.default_rng(SEED)
5     if n_eval is None:
6         n_eval = CFG.eval_points
7
8     X_eval, Y_eval = sample_solution_points(solution, n_eval, rng=rng)
9     X_t = to_tensor(X_eval, requires_grad=False)
10    Y_true = to_tensor(Y_eval, requires_grad=False)
11
12    model.eval()
13    with torch.no_grad():
14        Y_pred = model(X_t)
15        num = torch.linalg.norm(Y_pred - Y_true)
16        den = torch.linalg.norm(Y_true)

```

```

17     return float((num / den).detach().cpu())
18
19
20 def alpha_error(alpha_hat, alpha_true):
21     return abs(float(alpha_hat) - alpha_true) / alpha_true
22
23
24 def nu_log_error(nu_hat, nu_true, eps=1e-8):
25     nu_hat = max(float(nu_hat), 0.0)
26     return abs(np.log10(nu_hat + eps) - np.log10(nu_true + eps)) / abs(np.log10(nu_true + eps)
    )

```

Listing 7: Métricas de error usadas para comparar con el paper.

El bloque de métricas traduce las definiciones matemáticas del paper al código. La función `relative_solution_error` evalúa la red en puntos de prueba y compara la norma del error con la norma de la solución verdadera. Las funciones `alpha_error` y `nu_log_error` calculan errores escalares de parámetros.

5.8 Entrenamiento guiado por datos

```

1  # ===== Celda 33 del notebook =====
2  def train_data_driven(solution, n_data, cfg: RunConfig, seed=SEED):
3      rng = np.random.default_rng(seed)
4      model = HardBoundaryPINN(width=cfg.width, depth=cfg.depth).to(DEVICE)
5      optimizer = torch.optim.Adam(model.parameters(), lr=cfg.dd_lr0)
6
7      X_data_all, Y_data_all = sample_solution_points(solution, n_data, rng=rng)
8      X_ic_all, Y_ic_all = sample_initial_points(max(cfg.dd_batch_ic, 2048 if RUN_MODE == 'paper
9          ' else cfg.dd_batch_ic), rng=rng)
10
11     best_state = copy.deepcopy(model.state_dict())
12     best_data_loss = float('inf')
13     history = []
14
15     for step in range(1, cfg.dd_steps + 1):
16         lr = paper_lr(step, cfg.dd_lr0, cfg.dd_lr_min)
17         set_optimizer_lr(optimizer, lr)
18
19         bd = min(cfg.dd_batch_data, len(X_data_all))
20         idx_d = rng.choice(len(X_data_all), size=bd, replace=False)
21         bi = min(cfg.dd_batch_ic, len(X_ic_all))
22         idx_i = rng.choice(len(X_ic_all), size=bi, replace=False)
23
24         Xd = to_tensor(X_data_all[idx_d])
25         Yd = to_tensor(Y_data_all[idx_d])
26         Xi = to_tensor(X_ic_all[idx_i])
27         Yi = to_tensor(Y_ic_all[idx_i])
28
29         pred_d = model(Xd)
30         pred_i = model(Xi)
31         loss_data = ((pred_d - Yd)**2).mean()
32         loss_ic = ((pred_i - Yi)**2).mean()
33         loss = loss_data + loss_ic
34
35         optimizer.zero_grad()
36         loss.backward()
37         optimizer.step()
38
39         if float(loss_data.detach().cpu()) < best_data_loss:
40             best_data_loss = float(loss_data.detach().cpu())
41             best_state = copy.deepcopy(model.state_dict())
42
43         if step % cfg.print_every == 0 or step == 1:
44             history.append({'step': step, 'loss': float(loss.detach().cpu()),
45                 'loss_data': float(loss_data.detach().cpu()),

```

```

45         'loss_ic': float(loss_ic.detach().cpu()), 'lr': lr})
46
47     model.load_state_dict(best_state)
48
49     Xp = to_tensor(sample_pde_points(cfg.dd_batch_pde, cfg.t_final, rng=rng), requires_grad=
50         True)
51     alpha_hat, nu_hat = estimate_alpha_nu_least_squares(model, Xp)
52
53     rel_err = relative_solution_error(model, solution, n_eval=cfg.eval_points, rng=rng)
54
55     row = {
56         'method': 'Data-based fitting',
57         'pde_optimizer': 'least_squares_after_training',
58         'n_data': n_data,
59         'alpha_true': solution['alpha'],
60         'nu_true': solution['nu'],
61         'alpha_hat': float(alpha_hat.cpu()),
62         'nu_hat': float(nu_hat.cpu()),
63         'alpha_error': alpha_error(float(alpha_hat.cpu()), solution['alpha']),
64         'nu_error': nu_log_error(float(nu_hat.cpu()), solution['nu']),
65         'relative_error': rel_err,
66         'alpha_over_nu': solution['alpha'] / solution['nu'],
67     }
68     return row, pd.DataFrame(history), model

```

Listing 8: Baseline data-driven: entrenamiento con datos e IC, estimación PDE posterior.

Este bloque implementa el Algoritmo 1 del paper. La red aprende primero a imitar datos observados y condición inicial. Solo después se congelan sus pesos y se calculan α y ν con el residuo PDE. Esta separación permite responder una pregunta experimental: ¿basta con ajustar datos para identificar correctamente la ecuación física?

5.9 Entrenamiento PINN inverso

```

1  # ===== Celda 35 del notebook =====
2  def train_inverse_pinn(solution, n_data, pde_optimizer='newton', cfg: RunConfig = CFG, seed=
3      SEED):
4      rng = np.random.default_rng(seed)
5      model = HardBoundaryPINN(width=cfg.width, depth=cfg.depth).to(DEVICE)
6
7      X_data_all, Y_data_all = sample_solution_points(solution, n_data, rng=rng)
8      X_ic_all, Y_ic_all = sample_initial_points(max(cfg.pinn_batch_ic, 2048 if RUN_MODE == '
9          paper' else cfg.pinn_batch_ic), rng=rng)
10
11     # Inicializaciones del paper: alpha ~ U[0,5], nu ~ U[0,0.5].
12     alpha_param = nn.Parameter(torch.tensor(rng.uniform(0.0, 5.0), dtype=torch.float64, device=
13         DEVICE))
14     nu_param = nn.Parameter(torch.tensor(rng.uniform(0.0, 0.5), dtype=torch.float64, device=
15         DEVICE))
16
17     opt_theta = torch.optim.Adam(model.parameters(), lr=cfg.pinn_lr0)
18     if pde_optimizer == 'adam':
19         opt_phi = torch.optim.Adam([alpha_param, nu_param], lr=1e-3)
20         phi_interval = 10
21     elif pde_optimizer == 'newton':
22         opt_phi = None
23         phi_interval = 100
24     else:
25         raise ValueError("pde_optimizer debe ser 'adam' o 'newton'.")
26
27     best_state = copy.deepcopy(model.state_dict())
28     best_alpha = float(alpha_param.detach().cpu())
29     best_nu = float(nu_param.detach().cpu())
30     best_loss = float('inf')
31     history = []

```

```

29     for step in range(1, cfg.pinn_steps + 1):
30         lr = paper_lr(step, cfg.pinn_lr0, cfg.pinn_lr_min)
31         set_optimizer_lr(opt_theta, lr)
32
33         # El paper usa todos los datos disponibles para L_data en PINN.
34         Xd = to_tensor(X_data_all)
35         Yd = to_tensor(Y_data_all)
36
37         bi = min(cfg.pinn_batch_ic, len(X_ic_all))
38         idx_i = rng.choice(len(X_ic_all), size=bi, replace=False)
39         Xi = to_tensor(X_ic_all[idx_i])
40         Yi = to_tensor(Y_ic_all[idx_i])
41
42         Xp = to_tensor(sample_pde_points(cfg.pinn_batch_pde, cfg.t_final, rng=rng),
43                               requires_grad=True)
44
45         loss_pde = pde_loss(model, Xp, alpha_param, nu_param)
46         loss_data = ((model(Xd) - Yd)**2).mean()
47         loss_ic = ((model(Xi) - Yi)**2).mean()
48         loss = loss_pde + 10.0 * loss_data + 10.0 * loss_ic
49
50         opt_theta.zero_grad()
51         if opt_phi is not None:
52             opt_phi.zero_grad()
53
54         loss.backward()
55         opt_theta.step()
56
57         if pde_optimizer == 'adam' and step % phi_interval == 0:
58             opt_phi.step()
59             with torch.no_grad():
60                 nu_param.clamp_(min=0.0)
61
62         if pde_optimizer == 'newton' and step % phi_interval == 0:
63             Xp_newton = to_tensor(sample_pde_points(cfg.pinn_batch_pde, cfg.t_final, rng=rng),
64                                   requires_grad=True)
65             alpha_hat, nu_hat = estimate_alpha_nu_least_squares(model, Xp_newton)
66             with torch.no_grad():
67                 alpha_param.copy_(alpha_hat)
68                 nu_param.copy_(torch.clamp(nu_hat, min=0.0))
69
70         current_loss = float(loss.detach().cpu())
71         if current_loss < best_loss:
72             best_loss = current_loss
73             best_state = copy.deepcopy(model.state_dict())
74             best_alpha = float(alpha_param.detach().cpu())
75             best_nu = float(nu_param.detach().cpu())
76
77         if step % cfg.print_every == 0 or step == 1:
78             history.append({
79                 'step': step,
80                 'loss': current_loss,
81                 'loss_pde': float(loss_pde.detach().cpu()),
82                 'loss_data': float(loss_data.detach().cpu()),
83                 'loss_ic': float(loss_ic.detach().cpu()),
84                 'alpha_hat': float(alpha_param.detach().cpu()),
85                 'nu_hat': float(nu_param.detach().cpu()),
86                 'lr': lr,
87             })
88
89     model.load_state_dict(best_state)
90     rel_err = relative_solution_error(model, solution, n_eval=cfg.eval_points, rng=rng)
91
92     row = {
93         'method': 'PINN',
94         'pde_optimizer': pde_optimizer,
95         'n_data': n_data,
96         'alpha_true': solution['alpha'],
97         'nu_true': solution['nu'],

```



```

96     'alpha_hat': best_alpha,
97     'nu_hat': best_nu,
98     'alpha_error': alpha_error(best_alpha, solution['alpha']),
99     'nu_error': nu_log_error(best_nu, solution['nu']),
100     'relative_error': rel_err,
101     'alpha_over_nu': solution['alpha'] / solution['nu'],
102 }
103 return row, pd.DataFrame(history), model

```

Listing 9: Entrenamiento PINN inverso con Adam para la red y Adam/Newton para parámetros PDE.

La PINN inversa entrena simultáneamente la red y los parámetros. En modo `adam`, α y ν se tratan como parámetros entrenables de PyTorch. En modo `newton`, la red se actualiza con Adam y, cada cierto número de pasos, se recalculan α y ν con mínimos cuadrados.

- **Inicialización aleatoria.** El paper inicializa $\alpha \sim U[0, 5]$ y $\nu \sim U[0, 0.5]$. El notebook sigue esa idea.
- **Pérdida total.** La fórmula implementada coincide con (11).
- **Checkpoint.** El código guarda el mejor estado de la red según la menor pérdida. Esto evita quedarse con el último modelo si el entrenamiento empeora al final.
- **Intervalos.** En Newton no se recalculan parámetros en cada paso, sino cada `phi_interval`; eso reduce costo.

5.10 Grilla experimental

```

1  # ===== Celda 40 del notebook =====
2  def parameter_pairs_for_run(run_mode):
3      if run_mode == 'smoke':
4          return [(2.0, 1e-4)]
5      return [(row['alpha'], row['nu']) for _, row in params_df.iterrows()]
6
7
8  def data_sizes_for_run(run_mode):
9      if run_mode == 'paper':
10         pinn_sizes = [2048, 8192, 32768]
11         dd_sizes = [128, 512, 2048, 8192, 32768, 131072]
12     elif run_mode == 'demo':
13         pinn_sizes = [512, 2048]
14         dd_sizes = [128, 512, 2048]
15     else:
16         pinn_sizes = [256]
17         dd_sizes = [128]
18     return pinn_sizes, dd_sizes
19
20
21 def append_result(row, path=RESULTS_PATH):
22     df_row = pd.DataFrame([row])
23     if path.exists():
24         old = pd.read_csv(path)
25         new = pd.concat([old, df_row], ignore_index=True)
26     else:
27         new = df_row
28     new.to_csv(path, index=False)
29
30
31 def run_full_experiment(cfg: RunConfig, run_mode=RUN_MODE):
32     pinn_sizes, dd_sizes = data_sizes_for_run(run_mode)
33     param_pairs = parameter_pairs_for_run(run_mode)
34
35     all_rows = []

```

```

36     for alpha, nu in param_pairs:
37         solution = solve_burgers2d_fipy(alpha=alpha, nu=nu, cfg=cfg, force=False, verbose=True
38         )
39
40         for n_data in dd_sizes:
41             print(f'\n[Data-driven] alpha={alpha}, nu={nu:g}, n_data={n_data}')
42             row, hist, _ = train_data_driven(solution, n_data=n_data, cfg=cfg,
43                                             seed=SEED + int(n_data) + int(1000*alpha))
44             append_result(row)
45             all_rows.append(row)
46
47         for n_data in pinn_sizes:
48             for opt_name in ['newton', 'adam']:
49                 print(f'\n[PINN] alpha={alpha}, nu={nu:g}, n_data={n_data}, opt={opt_name}')
50                 row, hist, _ = train_inverse_pinn(solution, n_data=n_data,
51                                                  pde_optimizer=opt_name, cfg=cfg,
52                                                  seed=SEED + int(n_data) + int(1000*alpha))
53                 append_result(row)
54                 all_rows.append(row)
55
56     return pd.DataFrame(all_rows)
57
58 if RUN_FULL_GRID:
59     if RESULTS_PATH.exists():
60         print('Ya existe results_df.csv. Se anadiran nuevas filas si corres de nuevo.')
61         results_df = run_full_experiment(CFG, RUN_MODE)
62 else:
63     print('RUN_FULL_GRID=False: no se lanza la grilla completa.')
64     print('Para generar Figuras 2--8 desde entrenamiento completo, pon RUN_FULL_GRID=True y
        vuelve a ejecutar esta celda.')

```

Listing 10: Ejecución de la grilla experimental.

La grilla combina valores de α , ν , tamaños de datos y métodos. En modo paper hay 10 configuraciones de parámetros, 3 tamaños de datos para PINNs y 6 para el baseline. En modo demo, el número de pasos y tamaños se reducen.

5.11 Gráficas de resultados

```

1  # ===== Celda 46 del notebook =====
2  def plot_figure2(results_df, save_path=None):
3      ncols = [2048, 8192, 32768]
4      y_rows = [
5          ('alpha_error', r'$\alpha$ error'),
6          ('nu_error', r'$\nu$ error'),
7          ('relative_error', 'Relative error')
8      ]
9
10     METHOD_COLORS = {
11         'PINN': 'tab:blue',
12         'Data-based fitting': 'tab:orange'
13     }
14
15     fig, axs = plt.subplots(3, 3, figsize=(8.2, 7.2), sharex=False)
16
17     for i, (y_col, y_label) in enumerate(y_rows):
18         for j, n in enumerate(ncols):
19             ax = axs[i, j]
20
21             d = results_df[
22                 (results_df['n_data'] == n) &
23                 (
24                     ((results_df['method'] == 'PINN') &
25                     (results_df['pde_optimizer'].astype(str).str.lower() == 'newton')) |
26                     (results_df['method'] == 'Data-based fitting')

```

```

27         )
28     ].copy()
29
30     # Solo datos validos para escala logaritmica
31     d = d[
32         (d['alpha_over_nu'] > 0) &
33         (d[y_col] > 0) &
34         np.isfinite(d['alpha_over_nu']) &
35         np.isfinite(d[y_col])
36     ]
37
38     for method in ['PINN', 'Data-based fitting']:
39         dm = d[d['method'] == method]
40
41         if len(dm) == 0:
42             continue
43
44         ax.scatter(
45             dm['alpha_over_nu'],
46             dm[y_col],
47             color=METHOD_COLORS[method],
48             marker='o',
49             s=35,
50             alpha=0.9,
51             edgecolors='none'
52         )
53
54         ax.axhline(0.1, color='black', linestyle='--', linewidth=1)
55
56         # Fijar limites positivos aunque el panel quede vacio
57         ax.set_xscale('log')
58         ax.set_yscale('log')
59         ax.set_xlim(5e1, 5e4)
60         ax.set_ylim(5e-3, 2)
61
62         ax.set_title(f'# data = {n}', fontsize=9)
63
64         if j == 0:
65             ax.set_ylabel(y_label)
66
67         if i == 2:
68             ax.set_xlabel(r'$\alpha/\nu$')
69
70     method_handles = [
71         Line2D([0], [0], marker='o', linestyle='None',
72               color='tab:blue', label='PINN'),
73         Line2D([0], [0], marker='o', linestyle='None',
74               color='tab:orange', label='Data-based fitting')
75     ]
76
77     fig.legend(
78         handles=method_handles,
79         title='Method',
80         loc='lower center',
81         ncol=2,
82         frameon=True
83     )
84
85     fig.suptitle('Figura 2 -- PINN Newton vs data-driven', y=1.02)
86
87     # Mas estable que tight_layout en este caso
88     fig.subplots_adjust(
89         left=0.08,
90         right=0.98,
91         top=0.90,
92         bottom=0.16,
93         wspace=0.30,
94         hspace=0.45
95     )

```

```

96
97     if save_path:
98         fig.savefig(save_path, dpi=300, bbox_inches='tight')
99
100     plt.show()
101
102
103 plot_figure2(results_df, CACHE_DIR / 'figure2.png')
104
105 # ===== Celda 48 del notebook =====
106 def plot_figure3(results_df, save_path=None):
107     ncols = [2048, 8192, 32768]
108     y_rows = [('relative_error', 'Relative error'),
109              ('alpha_error', r'$\alpha$ error'),
110              ('nu_error', r'$\nu$ error')]
111
112     dbase = results_df[results_df['method'] == 'Data-based fitting']
113     fig, axs = plt.subplots(3, 3, figsize=(8.4, 7.2))
114     for i, (y_col, y_label) in enumerate(y_rows):
115         for j, n in enumerate(ncols):
116             ax = axs[i, j]
117             d = dbase[dbase['n_data'] == n]
118             _scatter_error(ax, d, y_col, title=f'# data = {n}')
119             if j == 0:
120                 ax.set_ylabel(y_label)
121     add_alpha_nu_legend(fig, anchor=(1.02, 0.5))
122     fig.suptitle('Figure 3 -- Data-driven estimation', y=1.02)
123     fig.tight_layout()
124     if save_path:
125         fig.savefig(save_path, dpi=300, bbox_inches='tight')
126     plt.show()
127
128 plot_figure3(results_df, CACHE_DIR / 'figure3.png')
129
130
131 # ===== Celda 50 del notebook =====
132 def plot_figure4(results_df, save_path=None):
133     dbase = results_df[results_df['method'] == 'PINN']
134     optimizers = ['newton', 'adam']
135     ncols = [2048, 8192, 32768]
136     y_groups = [('nu_error', r'$\nu$ error'), ('alpha_error', r'$\alpha$ error')]
137
138     fig, axs = plt.subplots(2, 6, figsize=(13.0, 5.2), sharex=False)
139     for row_i, opt in enumerate(optimizers):
140         for block_i, (y_col, y_label) in enumerate(y_groups):
141             for j, n in enumerate(ncols):
142                 ax = axs[row_i, block_i*3 + j]
143                 d = dbase[(dbase['pde_optimizer'] == opt) & (dbase['n_data'] == n)]
144                 _scatter_error(ax, d, y_col, title=f'{opt}; # data = {n}')
145                 if block_i == 0 and j == 0:
146                     ax.set_ylabel(y_label)
147                 if block_i == 1 and j == 0:
148                     ax.set_ylabel(y_label)
149     add_alpha_nu_legend(fig, anchor=(1.01, 0.5))
150     fig.suptitle('Figure 4 -- PINN parameter estimation errors', y=1.02)
151     fig.tight_layout()
152     if save_path:
153         fig.savefig(save_path, dpi=300, bbox_inches='tight')
154     plt.show()
155
156 plot_figure4(results_df, CACHE_DIR / 'figure4.png')
157
158
159 # ===== Celda 52 del notebook =====
160 def plot_figure5(results_df, save_path=None):
161     dbase = results_df[results_df['method'] == 'PINN']
162     optimizers = ['newton', 'adam']
163     ncols = [2048, 8192, 32768]

```

```

165 fig, axs = plt.subplots(2, 3, figsize=(8.0, 5.6))
166 for i, opt in enumerate(optimizers):
167     for j, n in enumerate(ncols):
168         ax = axs[i, j]
169         d = dbase[(dbase['pde_optimizer'] == opt) & (dbase['n_data'] == n)]
170         _scatter_error(ax, d, 'relative_error', title=f'{opt}; # data = {n}')
171         if j == 0:
172             ax.set_ylabel('Relative error')
173         add_alpha_nu_legend(fig, anchor=(1.02, 0.5))
174     fig.suptitle('Figure 5 -- PINN solution relative error', y=1.02)
175     fig.tight_layout()
176     if save_path:
177         fig.savefig(save_path, dpi=300, bbox_inches='tight')
178     plt.show()
179
180 plot_figure5(results_df, CACHE_DIR / 'figure5.png')
181
182
183 # ===== Celda 54 del notebook =====
184 def plot_data_driven_grid(results_df, y_col, ylabel, title, save_path=None):
185     dbase = results_df[results_df['method'] == 'Data-based fitting']
186     ncols = [128, 512, 2048, 8192, 32768, 131072]
187     fig, axs = plt.subplots(2, 3, figsize=(8.0, 5.8))
188     axs = axs.ravel()
189     for ax, n in zip(axs, ncols):
190         d = dbase[dbase['n_data'] == n]
191         _scatter_error(ax, d, y_col, title=f'# data = {n}')
192         ax.set_ylabel(ylabel)
193     add_alpha_nu_legend(fig, anchor=(1.02, 0.5))
194     fig.suptitle(title, y=1.02)
195     fig.tight_layout()
196     if save_path:
197         fig.savefig(save_path, dpi=300, bbox_inches='tight')
198     plt.show()
199
200 plot_data_driven_grid(results_df, 'relative_error', 'Relative error',
201                       'Figure 6 -- Data-driven solution error', CACHE_DIR / 'figure6.png')
202 plot_data_driven_grid(results_df, 'alpha_error', r'$\alpha$ error',
203                       'Figure 7 -- Data-driven alpha error', CACHE_DIR / 'figure7.png')
204 plot_data_driven_grid(results_df, 'nu_error', r'$\nu$ error',
205                       'Figure 8 -- Data-driven nu error', CACHE_DIR / 'figure8.png')

```

Listing 11: Funciones de graficación para Figuras 2–8.

Las funciones de graficación usan ejes logarítmicos porque los errores y la razón α/ν cubren varios órdenes de magnitud. La línea punteada en 10^{-1} marca el umbral de 10% usado por el paper para considerar exitosa la estimación.

6 Conexión directa entre código y teoría

Tabla 4: Traducción entre fórmulas del paper y bloques del notebook.

Concepto teórico	Objeto en el notebook	Comentario
$\mathbf{u}_t + \alpha(\mathbf{u} \cdot \nabla)\mathbf{u} - \nu\Delta\mathbf{u} = 0$	pde_pieces, pde_loss	El residuo se calcula con derivadas automáticas y se penaliza con error cuadrático.
Condición inicial	initial_u_np, initial_v_np, sample_initial_points	Genera los datos en $t = 0$ que entran en L_{IC} .
Frontera homogénea	HardBoundaryPINN	El factor $x(1-x)y(1-y)$ impone frontera cero por construcción.

Concepto teórico	Objeto en el notebook	Comentario
Parámetros α, ν	<code>alpha_param, nu_param</code>	Se estiman con Adam o con mínimos cuadrados/Newton.
$L_{PDE} + L_{BC} + L_{Data}$	<code>loss_pde, loss_data, loss_ic</code>	El notebook reemplaza L_{BC} por una frontera dura y usa L_{IC} .
Baseline data-driven	<code>train_data_driven</code>	Entrena sin residuo PDE y estima parámetros al final.
Métrica E_{rel}	<code>relative_solution_error</code>	Evalúa la reconstrucción de la solución.
Error logarítmico de ν	<code>nu_log_error</code>	Adecuado porque ν varía entre 10^{-4} y 10^{-2} .

7 Notas para un lector con poca programación

Variable. Nombre que guarda un valor. Por ejemplo, `alpha_param` guarda una estimación de α .

Función. Bloque reusable de instrucciones. Por ejemplo, `sample_solution_points` toma una solución y devuelve puntos muestreados.

Clase. Plantilla que agrupa datos y operaciones. `HardBoundaryPINN` es una clase que representa la red neuronal completa.

Tensor. Arreglo numérico usado por PyTorch. Puede vivir en CPU o GPU y puede recordar operaciones para calcular derivadas.

Batch. Subconjunto de puntos usado en una actualización. No siempre se usan todos los datos en cada paso, porque sería costoso.

Pérdida. Número que mide qué tan mal va el modelo. Entrenar significa modificar parámetros para reducir esa pérdida.

Optimizador. Método que actualiza parámetros. Adam es robusto para muchos pesos de red; Newton/mínimos cuadrados es mejor cuando el subproblema tiene estructura cuadrática.

Derivada automática. Técnica para calcular derivadas exactas del código computacional mediante regla de la cadena.

GPU. Procesador especializado que acelera operaciones matriciales. No cambia la matemática, solo el tiempo de ejecución.

8 Limitaciones de la réplica del notebook

1. **Costo computacional.** La configuración paper implica muchas corridas. En el paper, la malla de referencia tiene $256 \times 256 \times 72$ puntos por simulación; en modo demo se usa una malla reducida.
2. **Diferencia de librerías.** El paper original usa `pinn-jax`, `jax`, `flax` y `optax`; el notebook didáctico usa PyTorch para facilitar la explicación y derivadas automáticas.
3. **Figuras incompletas en modo demo.** Algunas columnas de las figuras quedan vacías si no se ejecutan tamaños de datos 8192 y 32768.

4. **Sensibilidad del entrenamiento.** Los errores dependen de semilla aleatoria, hardware, tasa de aprendizaje, número de pasos y discretización de FiPy.
5. **Régimen difícil.** Tanto en el paper como en la réplica, el régimen con α/ν grande es más complicado porque el flujo presenta variaciones rápidas.

9 Conclusiones

El notebook reproduce de forma pedagógica la arquitectura experimental del paper: genera datos de Burgers 2D, construye una PINN con frontera dura, calcula residuos mediante derivadas automáticas y compara una estrategia puramente guiada por datos contra una PINN inversa. La pieza conceptual más importante es la separación entre los pesos de la red θ y los parámetros físicos $\phi = (\alpha, \nu)$: aunque ambos se ajustan durante el aprendizaje, no tienen la misma naturaleza ni la misma escala.

La implementación con PyTorch hace visible el mecanismo matemático de la PINN. Cada evaluación de la red produce u y v ; la derivada automática produce las derivadas necesarias; el residuo mide cuánto viola la PDE; y el optimizador modifica la red para reducir simultáneamente error de datos, condición inicial y residuo físico.

Desde el punto de vista científico, la conclusión principal del paper es que la regularización física ayuda a estimar parámetros con menos datos que una red entrenada solo con observaciones, aunque no elimina las dificultades de optimización ni los problemas en regímenes altamente no viscosos.

A Salidas textuales relevantes del notebook

```

1
2 Resultado de la corrida individual (celda 37):
3 {'method': 'PINN', 'pde_optimizer': 'newton', 'n_data': 256,
4  'alpha_true': 2.0, 'nu_true': 0.0001,
5  'alpha_hat': 1.1371744210622161,
6  'nu_hat': 0.028541280933851703,
7  'alpha_error': 0.43141278946889194,
8  'nu_error': 0.6138642109269637,
9  'relative_error': 0.7798436746354874,
10 'alpha_over_nu': 20000.0}
11
12 Promedios reproducidos en modo demo (celda 56):
13 PDE optimizer  # data  alpha error  Relative error  nu error
14 adam          512      1.060146    0.384525      0.337898
15 newton         512      0.813796    0.548233      0.342569
16 adam          2048     1.134802    0.979736      0.767024
17 newton         2048     7.300168    0.875054      0.552538
18
19 Tabla 2 publicada en el paper (celda 57):
20 PDE optimizer  # data  alpha error  Relative error  nu error
21 Adam           2048     0.080       0.152          0.093
22 Newton         2048     0.086       0.150          0.056
23 Adam           8192     0.085       0.139          0.097
24 Newton         8192     0.085       0.130          0.065
25 Adam          32768     0.067       0.139          0.101
26 Newton        32768     0.088       0.126          0.071

```

Listing 12: Resumen textual de salidas guardadas en el notebook.

B Código completo del notebook

El siguiente listado reúne todas las celdas de código del notebook en una sola vista, normalizada para impresión. Las celdas de texto explicativo del notebook no se incluyen aquí porque fueron

incorporadas en el análisis principal.

```

1  #####
2  # Celda 3
3  #####
4  # This cell installs fipy and other required libraries.
5  !pip -q install fipy pyang pandas matplotlib scipy tqdm
6
7  #####
8  # Celda 5
9  #####
10 import os
11 import time
12 import copy
13 import math
14 import json
15 import warnings
16 from dataclasses import dataclass
17 from pathlib import Path
18
19 import numpy as np
20 import pandas as pd
21 import matplotlib.pyplot as plt
22 from matplotlib.lines import Line2D
23 from tqdm.auto import tqdm
24
25 import torch
26 import torch.nn as nn
27
28 warnings.filterwarnings('ignore')
29
30 # Reproducibilidad
31 SEED = 1234
32 np.random.seed(SEED)
33 torch.manual_seed(SEED)
34
35 # Doble precision como en el paper.
36 torch.set_default_dtype(torch.float64)
37
38 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
39 print('PyTorch:', torch.__version__, '| dispositivo:', DEVICE)
40
41 # Estilo visual cercano al paper: fondo claro, rejilla suave, marcadores limpios.
42 try:
43     plt.style.use('seaborn-v0_8-darkgrid')
44 except Exception:
45     plt.style.use('default')
46
47 CACHE_DIR = Path('/content/pinns_burgers2d_paper_cache')
48 CACHE_DIR.mkdir(parents=True, exist_ok=True)
49
50 RESULTS_PATH = CACHE_DIR / 'results_df.csv'
51
52
53 #####
54 # Celda 7
55 #####
56 # Cambia aqui segun lo que quieras ejecutar.
57 RUN_MODE = "demo" # "smoke", "demo" o "paper"
58 RUN_FULL_GRID = True # True para lanzar toda la grilla experimental.
59
60 # En modo smoke/demo solo se hacen pocos pasos para comprobar el pipeline.
61 # En modo paper se activan los tamanos del articulo.
62
63
64 #####
65 # Celda 9
66 #####
67 # Tabla 1 del paper: configuraciones de parametros para generar ground truth.

```



```

68 PARAMETER_GRID = [
69     {'nu': 1e-4, 'alpha_values': [1.0, 1.5, 2.0]},
70     {'nu': 1e-3, 'alpha_values': [1.0, 1.5, 2.0]},
71     {'nu': 1e-2, 'alpha_values': [0.5, 1.0, 1.5, 2.0]},
72 ]
73
74 param_rows = []
75 for row in PARAMETER_GRID:
76     for a in row['alpha_values']:
77         param_rows.append({'nu': row['nu'], 'alpha': a, 'alpha/nu': a / row['nu']})
78 params_df = pd.DataFrame(param_rows)
79 params_df
80
81
82 #=====
83 # Celda 10
84 #=====
85 # Tabla 3: hiperparametros del metodo data-driven del paper.
86 table3 = pd.DataFrame([
87     ['Number of hidden units', 256],
88     ['Number of layers', 10],
89     ['Activation function', 'tanh'],
90     ['Batch size (Data loss)', 2048],
91     ['Batch size (IC loss)', 2048],
92     ['Batch size (PDE loss)', 8192],
93     ['lambda_Data', 1],
94     ['lambda_IC', 1],
95     ['Optimizer', 'Adam'],
96     ['Number of epochs', 50000],
97     ['Initial learning rate', '5e-3'],
98     ['Minimum learning rate', '1e-6'],
99     ['Exponential decay rate', 0.925],
100    ['Exponential decay interval', 5000],
101 ], columns=['Hyperparameter', 'Value'])
102
103 table3
104
105
106 #=====
107 # Celda 11
108 #=====
109 # Tabla 4: hiperparametros PINN del paper.
110 table4 = pd.DataFrame([
111     ['Number of hidden units', 256],
112     ['Number of layers', 10],
113     ['Activation function', 'tanh'],
114     ['Batch size (Data loss)', 'Varies'],
115     ['Batch size (IC loss)', 1024],
116     ['Batch size (PDE loss)', 2048],
117     ['lambda_PDE', 1],
118     ['lambda_Data', 10],
119     ['lambda_IC', 10],
120     ['NN Optimizer', 'Adam'],
121     ['PDE Optimizer', '{Adam, Newton}'],
122     ['Number of gradient steps', 100000],
123     ['Parameter estimation interval', '{10 Adam, 100 Newton}'],
124     ['Initial NN learning rate', '5e-3'],
125     ['Minimum NN learning rate', '1e-5'],
126     ['Exponential decay rate', 0.925],
127     ['Exponential decay interval', 5000],
128     ['PDE parameter step size', '1e-3'],
129 ], columns=['Hyperparameter', 'Value'])
130
131 table4
132
133
134 #=====
135 # Celda 13
136 #=====

```

```

137 @dataclass
138 class RunConfig:
139     # Datos de referencia
140     nx: int
141     ny: int
142     nt: int
143     t_final: float
144     nonlinear_sweeps: int
145
146     # Arquitectura
147     width: int
148     depth: int
149
150     # Entrenamiento data-driven
151     dd_steps: int
152     dd_batch_data: int
153     dd_batch_ic: int
154     dd_batch_pde: int
155     dd_lr0: float
156     dd_lr_min: float
157
158     # Entrenamiento PINN
159     pinn_steps: int
160     pinn_batch_ic: int
161     pinn_batch_pde: int
162     pinn_lr0: float
163     pinn_lr_min: float
164
165     # Evaluacion y logging
166     eval_points: int
167     print_every: int
168
169
170 def get_config(run_mode: str) -> RunConfig:
171     if run_mode == 'paper':
172         return RunConfig(
173             nx=256, ny=256, nt=72, t_final=0.499, nonlinear_sweeps=4,
174             width=256, depth=10,
175             dd_steps=50000, dd_batch_data=2048, dd_batch_ic=2048, dd_batch_pde=8192,
176             dd_lr0=5e-3, dd_lr_min=1e-6,
177             pinn_steps=100000, pinn_batch_ic=1024, pinn_batch_pde=2048,
178             pinn_lr0=5e-3, pinn_lr_min=1e-5,
179             eval_points=200000, print_every=5000,
180         )
181     if run_mode == 'demo':
182         return RunConfig(
183             nx=48, ny=48, nt=24, t_final=0.499, nonlinear_sweeps=2,
184             width=80, depth=4,
185             dd_steps=1500, dd_batch_data=512, dd_batch_ic=256, dd_batch_pde=512,
186             dd_lr0=3e-3, dd_lr_min=1e-5,
187             pinn_steps=1500, pinn_batch_ic=256, pinn_batch_pde=512,
188             pinn_lr0=3e-3, pinn_lr_min=1e-5,
189             eval_points=6500, print_every=250,
190         )
191     if run_mode == 'smoke':
192         return RunConfig(
193             nx=48, ny=48, nt=16, t_final=0.499, nonlinear_sweeps=1,
194             width=48, depth=3,
195             dd_steps=80, dd_batch_data=128, dd_batch_ic=128, dd_batch_pde=256,
196             dd_lr0=2e-3, dd_lr_min=1e-5,
197             pinn_steps=100, pinn_batch_ic=128, pinn_batch_pde=256,
198             pinn_lr0=2e-3, pinn_lr_min=1e-5,
199             eval_points=5000, print_every=25,
200         )
201     raise ValueError("RUN_MODE debe ser 'smoke', 'demo' o 'paper'.")
202
203 CFG = get_config(RUN_MODE)
204 CFG
205

```

```

206
207 #=====
208 # Celda 15
209 #=====
210 def initial_u_np(x, y):
211     """u(x,y,0) = sin(6*pi*y)*x*(1-x)."""
212     return np.sin(6*np.pi*y) * x * (1.0 - x)
213
214
215 def initial_v_np(x, y):
216     """v(x,y,0) = -sin(6*pi*x)*y*(1-y)."""
217     return -np.sin(6*np.pi*x) * y * (1.0 - y)
218
219
220 #=====
221 # Celda 17
222 #=====
223 def _solution_filename(alpha, nu, cfg: RunConfig):
224     tag = f"alpha_{alpha:g}_nu_{nu:.0e}_nx_{cfg.nx}_ny_{cfg.ny}_nt_{cfg.nt}_tf_{cfg.t_final}"
225     return CACHE_DIR / (tag.replace('.', 'p') + '.npz')
226
227
228 def _cell_values_to_image(phi, nx, ny):
229     """Convierte valores de celdas FiPy a imagen [ny, nx] para imshow."""
230     arr = np.asarray(phi.value, dtype=np.float64)
231     return arr.reshape((nx, ny), order='F').T
232
233
234 def solve_burgers2d_fipy(alpha=2.0, nu=1e-4, cfg: RunConfig = CFG,
235     force=False, verbose=True):
236     """
237     Genera una solucion de referencia para Burgers 2D vectorial usando FiPy.
238
239     Ecuaciones:
240         u_t + alpha * div( u * [u, v] ) = nu * Delta u
241         v_t + alpha * div( v * [u, v] ) = nu * Delta v
242
243     Esta es la forma conservativa usada para aproximar el termino no lineal.
244     """
245     path = _solution_filename(alpha, nu, cfg)
246     if path.exists() and not force:
247         data = np.load(path)
248         return {
249             'x': data['x'], 'y': data['y'], 't': data['t'],
250             'U': data['U'], 'V': data['V'],
251             'alpha': float(data['alpha']), 'nu': float(data['nu'])
252         }
253
254     from fipy import CellVariable, FaceVariable, Grid2D
255     from fipy import TransientTerm, DiffusionTerm, PowerLawConvectionTerm
256
257     try:
258         # Si pyAMG esta disponible, FiPy puede usarlo. Si no, cae al solver por defecto.
259         from fipy.solvers.pyAMG import LinearGMRESSolver
260         solver = LinearGMRESSolver(tolerance=1e-10, iterations=2000)
261     except Exception:
262         from fipy import LinearLUSolver
263         solver = LinearLUSolver(tolerance=1e-10, iterations=2000)
264
265     nx, ny, nt = cfg.nx, cfg.ny, cfg.nt
266     dx = 1.0 / nx
267     dy = 1.0 / ny
268     dt = cfg.t_final / (nt - 1)
269
270     mesh = Grid2D(dx=dx, dy=dy, nx=nx, ny=ny)
271     x_cc, y_cc = mesh.cellCenters
272     x_val = np.asarray(x_cc)
273     y_val = np.asarray(y_cc)
274

```

```

275 u = CellVariable(mesh=mesh, name='u', value=initial_u_np(x_val, y_val), hasOld=True)
276 v = CellVariable(mesh=mesh, name='v', value=initial_v_np(x_val, y_val), hasOld=True)
277
278 # Condicion de frontera homogenea en todo el borde exterior.
279 u.constrain(0.0, where=mesh.exteriorFaces)
280 v.constrain(0.0, where=mesh.exteriorFaces)
281
282 velocity = FaceVariable(mesh=mesh, rank=1)
283
284 eq_u = TransientTerm(var=u) + PowerLawConvectionTerm(coeff=velocity, var=u) ==
      DiffusionTerm(coeff=nu, var=u)
285 eq_v = TransientTerm(var=v) + PowerLawConvectionTerm(coeff=velocity, var=v) ==
      DiffusionTerm(coeff=nu, var=v)
286
287 x_grid = (np.arange(nx) + 0.5) * dx
288 y_grid = (np.arange(ny) + 0.5) * dy
289 t_grid = np.linspace(0.0, cfg.t_final, nt)
290
291 U = np.zeros((nt, ny, nx), dtype=np.float64)
292 V = np.zeros((nt, ny, nx), dtype=np.float64)
293 U[0] = _cell_values_to_image(u, nx, ny)
294 V[0] = _cell_values_to_image(v, nx, ny)
295
296 iterator = range(1, nt)
297 if verbose:
298     iterator = tqdm(iterator, desc=f'FiPy alpha={alpha}, nu={nu:g}')
299
300 for k in iterator:
301     u.updateOld()
302     v.updateOld()
303
304     for _ in range(cfg.nonlinear_sweeps):
305         # Velocidad convectiva de cara: alpha * (u, v).
306         velocity[:] = (alpha * u.arithmeticFaceValue, alpha * v.arithmeticFaceValue)
307         eq_u.sweep(var=u, dt=dt, solver=solver)
308         eq_v.sweep(var=v, dt=dt, solver=solver)
309
310         U[k] = _cell_values_to_image(u, nx, ny)
311         V[k] = _cell_values_to_image(v, nx, ny)
312
313 np.savez_compressed(path, x=x_grid, y=y_grid, t=t_grid, U=U, V=V,
314                    alpha=np.array(alpha), nu=np.array(nu))
315 return {'x': x_grid, 'y': y_grid, 't': t_grid, 'U': U, 'V': V,
316        'alpha': alpha, 'nu': nu}
317
318 =====
319 # Celda 19
320 =====
321 def nearest_time_index(t_grid, target):
322     return int(np.argmax(np.abs(t_grid - target)))
323
324
325 def get_paper_cmap():
326     # El paper visualmente se parece al colormap icefire.
327     try:
328         import seaborn as sns
329         return sns.color_palette('icefire', as_cmap=True)
330     except Exception:
331         return 'RdBu_r'
332
333
334 def plot_figure1(solution, save_path=None):
335     x, y, t = solution['x'], solution['y'], solution['t']
336     U = solution['U']
337     alpha, nu = solution['alpha'], solution['nu']
338
339     times = [0.00, 0.07, 0.14, 0.35]
340     idxs = [nearest_time_index(t, tau) for tau in times]
341

```

```

342     vmax = 0.35
343     vmin = -vmax
344     cmap = get_paper_cmap()
345
346     fig, axs = plt.subplots(2, 2, figsize=(7.2, 5.7), constrained_layout=True)
347     axs = axs.ravel()
348
349     for ax, tau, idx in zip(axs, times, idxs):
350         im = ax.imshow(U[idx], origin='lower', extent=[0, 1, 0, 1],
351                        cmap=cmap, vmin=vmin, vmax=vmax, interpolation='bilinear')
352         ax.set_title(f'time = {tau:.2f}', fontsize=12)
353         ax.set_xticks([])
354         ax.set_yticks([])
355         plt.colorbar(im, ax=ax, fraction=0.046, pad=0.03)
356
357     fig.suptitle(rf'$u(x,y,t)$ for $\alpha={alpha:g}$ and $\nu=10^{\{\int(np.log10(\nu))\}}$',
358                fontsize=15, y=1.05)
359     if save_path is not None:
360         fig.savefig(save_path, dpi=300, bbox_inches='tight')
361     plt.show()
362
363
364 sol_fig1 = solve_burgers2d_fipy(alpha=2.0, nu=1e-4, cfg=CFG, force=False, verbose=True)
365 plot_figure1(sol_fig1, CACHE_DIR / 'figure1_snapshots.png')
366
367
368 #=====
369 # Celda 21
370 #=====
371 def sample_solution_points(solution, n_points, rng=None):
372     """Muestra puntos etiquetados de la solución completa."""
373     if rng is None:
374         rng = np.random.default_rng(SEED)
375
376     x, y, t = solution['x'], solution['y'], solution['t']
377     U, V = solution['U'], solution['V']
378     nt, ny, nx = U.shape
379     total = nt * ny * nx
380     replace = n_points > total
381     idx = rng.choice(total, size=n_points, replace=replace)
382
383     it = idx // (ny * nx)
384     rem = idx % (ny * nx)
385     iy = rem // nx
386     ix = rem % nx
387
388     X = np.stack([x[ix], y[iy], t[it]], axis=1)
389     Y = np.stack([U[it, iy, ix], V[it, iy, ix]], axis=1)
390     return X.astype(np.float64), Y.astype(np.float64)
391
392
393 def sample_initial_points(n_points, rng=None):
394     """Muestra puntos de condición inicial t=0."""
395     if rng is None:
396         rng = np.random.default_rng(SEED)
397     x = rng.random(n_points)
398     y = rng.random(n_points)
399     t = np.zeros(n_points)
400     U0 = initial_u_np(x, y)
401     V0 = initial_v_np(x, y)
402     X = np.stack([x, y, t], axis=1)
403     Y = np.stack([U0, V0], axis=1)
404     return X.astype(np.float64), Y.astype(np.float64)
405
406
407 def sample_pde_points(n_points, t_final=0.499, rng=None):
408     """Puntos interiores para el residuo físico."""
409     if rng is None:
410         rng = np.random.default_rng(SEED)

```

```

411     x = rng.random(n_points)
412     y = rng.random(n_points)
413     t = t_final * rng.random(n_points)
414     X = np.stack([x, y, t], axis=1)
415     return X.astype(np.float64)
416
417
418 def to_tensor(a, requires_grad=False):
419     z = torch.tensor(a, dtype=torch.float64, device=DEVICE)
420     return z.requires_grad_(requires_grad)
421
422
423 #####
424 # Celda 23
425 #####
426 class EDMLP(nn.Module):
427     """
428     Encoder-Decoder MLP tipo modified MLP.
429
430     Esta version usa compuertas suaves entre dos codificaciones U y V.
431     Mantiene el espiritu de la ED-MLP usada en PINNs modernos y es suficiente
432     para reproducir la estructura experimental del paper de forma didactica.
433     """
434     def __init__(self, in_dim=3, out_dim=2, width=256, depth=10):
435         super().__init__()
436         self.U = nn.Linear(in_dim, width)
437         self.V = nn.Linear(in_dim, width)
438         self.first = nn.Linear(in_dim, width)
439         self.hidden = nn.ModuleList([nn.Linear(width, width) for _ in range(depth - 1)])
440         self.last = nn.Linear(width, out_dim)
441         self.act = nn.Tanh()
442         self.reset_parameters()
443
444     def reset_parameters(self):
445         for m in self.modules():
446             if isinstance(m, nn.Linear):
447                 nn.init.xavier_normal_(m.weight)
448                 nn.init.zeros_(m.bias)
449
450     def forward(self, X):
451         U = self.act(self.U(X))
452         V = self.act(self.V(X))
453         H = self.act(self.first(X))
454         for layer in self.hidden:
455             Z = self.act(layer(H))
456             H = (1.0 - Z) * U + Z * V
457         return self.last(H)
458
459
460 class HardBoundaryPINN(nn.Module):
461     """PINN que satisface u=v=0 en la frontera espacial por construccion."""
462     def __init__(self, width=256, depth=10):
463         super().__init__()
464         self.net = EDMLP(in_dim=3, out_dim=2, width=width, depth=depth)
465
466     def forward(self, X):
467         raw = self.net(X)
468         x = X[:, 0:1]
469         y = X[:, 1:2]
470         factor = x * (1.0 - x) * y * (1.0 - y)
471         return factor * raw
472
473
474 #####
475 # Celda 25
476 #####
477 def grad(outputs, inputs):
478     return torch.autograd.grad(
479         outputs, inputs,

```

```

480         grad_outputs=torch.ones_like(outputs),
481         create_graph=True,
482         retain_graph=True,
483         only_inputs=True,
484     )[0]
485
486
487 def pde_pieces(model, X):
488     """
489     Devuelve las piezas del residuo:
490     u_t, v_t, A_u, A_v, D_u, D_v.
491     """
492     X = X.clone().detach().requires_grad_(True)
493     pred = model(X)
494     u = pred[:, 0:1]
495     v = pred[:, 1:2]
496
497     gu = grad(u, X)
498     gv = grad(v, X)
499
500     u_x, u_y, u_t = gu[:, 0:1], gu[:, 1:2], gu[:, 2:3]
501     v_x, v_y, v_t = gv[:, 0:1], gv[:, 1:2], gv[:, 2:3]
502
503     u_xx = grad(u_x, X)[:, 0:1]
504     u_yy = grad(u_y, X)[:, 1:2]
505     v_xx = grad(v_x, X)[:, 0:1]
506     v_yy = grad(v_y, X)[:, 1:2]
507
508     A_u = u * u_x + v * u_y
509     A_v = u * v_x + v * v_y
510     D_u = u_xx + u_yy
511     D_v = v_xx + v_yy
512
513     return u_t, v_t, A_u, A_v, D_u, D_v
514
515
516 def pde_residual(model, X, alpha, nu):
517     u_t, v_t, A_u, A_v, D_u, D_v = pde_pieces(model, X)
518     r_u = u_t + alpha * A_u - nu * D_u
519     r_v = v_t + alpha * A_v - nu * D_v
520     return r_u, r_v
521
522
523 def pde_loss(model, X, alpha, nu):
524     r_u, r_v = pde_residual(model, X, alpha, nu)
525     return (r_u**2).mean() + (r_v**2).mean()
526
527
528 #=====
529 # Celda 27
530 #=====
531 @torch.no_grad()
532 def _safe_scalar(x):
533     if isinstance(x, torch.Tensor):
534         return float(x.detach().cpu().item())
535     return float(x)
536
537
538 def estimate_alpha_nu_least_squares(model, X_pde, ridge=1e-12, clip_nu=True):
539     """Estimacion cerrada de alpha y nu para red fija."""
540     u_t, v_t, A_u, A_v, D_u, D_v = pde_pieces(model, X_pde)
541
542     b = torch.cat([u_t, v_t], dim=0) # [2N, 1]
543     col_alpha = torch.cat([A_u, A_v], dim=0) # [2N, 1]
544     col_nu = torch.cat([-D_u, -D_v], dim=0) # [2N, 1]
545     M = torch.cat([col_alpha, col_nu], dim=1) # [2N, 2]
546
547     # Min || b + M phi ||^2 => (M^T M) phi = - M^T b.
548     A = M.T @ M + ridge * torch.eye(2, dtype=torch.float64, device=DEVICE)

```

```

549     rhs = -M.T @ b
550     phi = torch.linalg.solve(A, rhs).flatten()
551
552     alpha_hat = phi[0]
553     nu_hat = phi[1]
554     if clip_nu:
555         nu_hat = torch.clamp(nu_hat, min=0.0)
556
557     return alpha_hat.detach(), nu_hat.detach()
558
559
560 #=====
561 # Celda 29
562 #=====
563 def paper_lr(step, lr0, lr_min, decay_rate=0.925, decay_interval=5000):
564     lr = lr0 * (decay_rate ** (step // decay_interval))
565     return max(lr, lr_min)
566
567
568 def set_optimizer_lr(optimizer, lr):
569     for group in optimizer.param_groups:
570         group['lr'] = lr
571
572
573 #=====
574 # Celda 31
575 #=====
576 def relative_solution_error(model, solution, n_eval=None, rng=None):
577     if rng is None:
578         rng = np.random.default_rng(SEED)
579     if n_eval is None:
580         n_eval = CFG.eval_points
581
582     X_eval, Y_eval = sample_solution_points(solution, n_eval, rng=rng)
583     X_t = to_tensor(X_eval, requires_grad=False)
584     Y_true = to_tensor(Y_eval, requires_grad=False)
585
586     model.eval()
587     with torch.no_grad():
588         Y_pred = model(X_t)
589         num = torch.linalg.norm(Y_pred - Y_true)
590         den = torch.linalg.norm(Y_true)
591     return float((num / den).detach().cpu())
592
593
594 def alpha_error(alpha_hat, alpha_true):
595     return abs(float(alpha_hat) - alpha_true) / alpha_true
596
597
598 def nu_log_error(nu_hat, nu_true, eps=1e-8):
599     nu_hat = max(float(nu_hat), 0.0)
600     return abs(np.log10(nu_hat + eps) - np.log10(nu_true + eps)) / abs(np.log10(nu_true + eps))
601
602
603 #=====
604 # Celda 33
605 #=====
606 def train_data_driven(solution, n_data, cfg: RunConfig, seed=SEED):
607     rng = np.random.default_rng(seed)
608     model = HardBoundaryPINN(width=cfg.width, depth=cfg.depth).to(DEVICE)
609     optimizer = torch.optim.Adam(model.parameters(), lr=cfg.dd_lr0)
610
611     X_data_all, Y_data_all = sample_solution_points(solution, n_data, rng=rng)
612     X_ic_all, Y_ic_all = sample_initial_points(max(cfg.dd_batch_ic, 2048 if RUN_MODE == 'paper'
613         ' else cfg.dd_batch_ic), rng=rng)
614
615     best_state = copy.deepcopy(model.state_dict())
616     best_data_loss = float('inf')

```



```

616     history = []
617
618     for step in range(1, cfg.dd_steps + 1):
619         lr = paper_lr(step, cfg.dd_lr0, cfg.dd_lr_min)
620         set_optimizer_lr(optimizer, lr)
621
622         bd = min(cfg.dd_batch_data, len(X_data_all))
623         idx_d = rng.choice(len(X_data_all), size=bd, replace=False)
624         bi = min(cfg.dd_batch_ic, len(X_ic_all))
625         idx_i = rng.choice(len(X_ic_all), size=bi, replace=False)
626
627         Xd = to_tensor(X_data_all[idx_d])
628         Yd = to_tensor(Y_data_all[idx_d])
629         Xi = to_tensor(X_ic_all[idx_i])
630         Yi = to_tensor(Y_ic_all[idx_i])
631
632         pred_d = model(Xd)
633         pred_i = model(Xi)
634         loss_data = ((pred_d - Yd)**2).mean()
635         loss_ic = ((pred_i - Yi)**2).mean()
636         loss = loss_data + loss_ic
637
638         optimizer.zero_grad()
639         loss.backward()
640         optimizer.step()
641
642         if float(loss_data.detach().cpu()) < best_data_loss:
643             best_data_loss = float(loss_data.detach().cpu())
644             best_state = copy.deepcopy(model.state_dict())
645
646         if step % cfg.print_every == 0 or step == 1:
647             history.append({'step': step, 'loss': float(loss.detach().cpu()),
648                           'loss_data': float(loss_data.detach().cpu()),
649                           'loss_ic': float(loss_ic.detach().cpu()), 'lr': lr})
650
651     model.load_state_dict(best_state)
652
653     Xp = to_tensor(sample_pde_points(cfg.dd_batch_pde, cfg.t_final, rng=rng), requires_grad=
        True)
654     alpha_hat, nu_hat = estimate_alpha_nu_least_squares(model, Xp)
655
656     rel_err = relative_solution_error(model, solution, n_eval=cfg.eval_points, rng=rng)
657
658     row = {
659         'method': 'Data-based fitting',
660         'pde_optimizer': 'least_squares_after_training',
661         'n_data': n_data,
662         'alpha_true': solution['alpha'],
663         'nu_true': solution['nu'],
664         'alpha_hat': float(alpha_hat.cpu()),
665         'nu_hat': float(nu_hat.cpu()),
666         'alpha_error': alpha_error(float(alpha_hat.cpu()), solution['alpha']),
667         'nu_error': nu_log_error(float(nu_hat.cpu()), solution['nu']),
668         'relative_error': rel_err,
669         'alpha_over_nu': solution['alpha'] / solution['nu'],
670     }
671     return row, pd.DataFrame(history), model
672
673
674     =====
675     # Celda 35
676     =====
677     def train_inverse_pinn(solution, n_data, pde_optimizer='newton', cfg: RunConfig = CFG, seed=
        SEED):
678         rng = np.random.default_rng(seed)
679         model = HardBoundaryPINN(width=cfg.width, depth=cfg.depth).to(DEVICE)
680
681         X_data_all, Y_data_all = sample_solution_points(solution, n_data, rng=rng)
682         X_ic_all, Y_ic_all = sample_initial_points(max(cfg.pinn_batch_ic, 2048 if RUN_MODE == '

```

```

        paper' else cfg.pinn_batch_ic), rng=rng)
683
684 # Inicializaciones del paper:  $\alpha \sim U[0,5]$ ,  $\nu \sim U[0,0.5]$ .
685 alpha_param = nn.Parameter(torch.tensor(rng.uniform(0.0, 5.0), dtype=torch.float64, device
        =DEVICE))
686 nu_param = nn.Parameter(torch.tensor(rng.uniform(0.0, 0.5), dtype=torch.float64, device=
        DEVICE))
687
688 opt_theta = torch.optim.Adam(model.parameters(), lr=cfg.pinn_lr0)
689 if pde_optimizer == 'adam':
690     opt_phi = torch.optim.Adam([alpha_param, nu_param], lr=1e-3)
691     phi_interval = 10
692 elif pde_optimizer == 'newton':
693     opt_phi = None
694     phi_interval = 100
695 else:
696     raise ValueError("pde_optimizer debe ser 'adam' o 'newton'.")
697
698 best_state = copy.deepcopy(model.state_dict())
699 best_alpha = float(alpha_param.detach().cpu())
700 best_nu = float(nu_param.detach().cpu())
701 best_loss = float('inf')
702 history = []
703
704 for step in range(1, cfg.pinn_steps + 1):
705     lr = paper_lr(step, cfg.pinn_lr0, cfg.pinn_lr_min)
706     set_optimizer_lr(opt_theta, lr)
707
708     # El paper usa todos los datos disponibles para  $L_{data}$  en PINN.
709     Xd = to_tensor(X_data_all)
710     Yd = to_tensor(Y_data_all)
711
712     bi = min(cfg.pinn_batch_ic, len(X_ic_all))
713     idx_i = rng.choice(len(X_ic_all), size=bi, replace=False)
714     Xi = to_tensor(X_ic_all[idx_i])
715     Yi = to_tensor(Y_ic_all[idx_i])
716
717     Xp = to_tensor(sample_pde_points(cfg.pinn_batch_pde, cfg.t_final, rng=rng),
        requires_grad=True)
718
719     loss_pde = pde_loss(model, Xp, alpha_param, nu_param)
720     loss_data = ((model(Xd) - Yd)**2).mean()
721     loss_ic = ((model(Xi) - Yi)**2).mean()
722     loss = loss_pde + 10.0 * loss_data + 10.0 * loss_ic
723
724     opt_theta.zero_grad()
725     if opt_phi is not None:
726         opt_phi.zero_grad()
727
728     loss.backward()
729     opt_theta.step()
730
731     if pde_optimizer == 'adam' and step % phi_interval == 0:
732         opt_phi.step()
733         with torch.no_grad():
734             nu_param.clamp_(min=0.0)
735
736     if pde_optimizer == 'newton' and step % phi_interval == 0:
737         Xp_newton = to_tensor(sample_pde_points(cfg.pinn_batch_pde, cfg.t_final, rng=rng),
            requires_grad=True)
738         alpha_hat, nu_hat = estimate_alpha_nu_least_squares(model, Xp_newton)
739         with torch.no_grad():
740             alpha_param.copy_(alpha_hat)
741             nu_param.copy_(torch.clamp(nu_hat, min=0.0))
742
743     current_loss = float(loss.detach().cpu())
744     if current_loss < best_loss:
745         best_loss = current_loss
746         best_state = copy.deepcopy(model.state_dict())

```

```

747         best_alpha = float(alpha_param.detach().cpu())
748         best_nu = float(nu_param.detach().cpu())
749
750     if step % cfg.print_every == 0 or step == 1:
751         history.append({
752             'step': step,
753             'loss': current_loss,
754             'loss_pde': float(loss_pde.detach().cpu()),
755             'loss_data': float(loss_data.detach().cpu()),
756             'loss_ic': float(loss_ic.detach().cpu()),
757             'alpha_hat': float(alpha_param.detach().cpu()),
758             'nu_hat': float(nu_param.detach().cpu()),
759             'lr': lr,
760         })
761
762     model.load_state_dict(best_state)
763     rel_err = relative_solution_error(model, solution, n_eval=cfg.eval_points, rng=rng)
764
765     row = {
766         'method': 'PINN',
767         'pde_optimizer': pde_optimizer,
768         'n_data': n_data,
769         'alpha_true': solution['alpha'],
770         'nu_true': solution['nu'],
771         'alpha_hat': best_alpha,
772         'nu_hat': best_nu,
773         'alpha_error': alpha_error(best_alpha, solution['alpha']),
774         'nu_error': nu_log_error(best_nu, solution['nu']),
775         'relative_error': rel_err,
776         'alpha_over_nu': solution['alpha'] / solution['nu'],
777     }
778     return row, pd.DataFrame(history), model
779
780
781 #=====
782 # Celda 37
783 #=====
784 # Corrida de prueba: una configuracion pequena.
785 smoke_alpha = 2.0
786 smoke_nu = 1e-4
787 smoke_n_data = 2048 if RUN_MODE == 'paper' else 256
788
789 sol_test = solve_burgers2d_fipy(alpha=smoke_alpha, nu=smoke_nu, cfg=CFG, force=False, verbose=
    True)
790 row_test, hist_test, model_test = train_inverse_pinn(
791     sol_test, n_data=smoke_n_data, pde_optimizer='newton', cfg=CFG, seed=SEED
792 )
793
794 print(row_test)
795 hist_test.tail()
796
797
798 #=====
799 # Celda 38
800 #=====
801 # Curva de entrenamiento de la prueba.
802 plt.figure(figsize=(6.5, 3.8))
803 plt.semilogy(hist_test['step'], hist_test['loss'], label='loss total')
804 plt.semilogy(hist_test['step'], hist_test['loss_pde'], label='PDE')
805 plt.semilogy(hist_test['step'], hist_test['loss_data'], label='data')
806 plt.semilogy(hist_test['step'], hist_test['loss_ic'], label='IC')
807 plt.xlabel('step')
808 plt.ylabel('loss')
809 plt.title('Prueba individual -- evolucion de perdidas')
810 plt.legend()
811 plt.tight_layout()
812 plt.show()
813
814

```

```

815 #=====
816 # Celda 40
817 #=====
818 def parameter_pairs_for_run(run_mode):
819     if run_mode == 'smoke':
820         return [(2.0, 1e-4)]
821     return [(row['alpha'], row['nu']) for _, row in params_df.iterrows()]
822
823
824 def data_sizes_for_run(run_mode):
825     if run_mode == 'paper':
826         pinn_sizes = [2048, 8192, 32768]
827         dd_sizes = [128, 512, 2048, 8192, 32768, 131072]
828     elif run_mode == 'demo':
829         pinn_sizes = [512, 2048]
830         dd_sizes = [128, 512, 2048]
831     else:
832         pinn_sizes = [256]
833         dd_sizes = [128]
834     return pinn_sizes, dd_sizes
835
836
837 def append_result(row, path=RESULTS_PATH):
838     df_row = pd.DataFrame([row])
839     if path.exists():
840         old = pd.read_csv(path)
841         new = pd.concat([old, df_row], ignore_index=True)
842     else:
843         new = df_row
844     new.to_csv(path, index=False)
845
846
847 def run_full_experiment(cfg: RunConfig, run_mode=RUN_MODE):
848     pinn_sizes, dd_sizes = data_sizes_for_run(run_mode)
849     param_pairs = parameter_pairs_for_run(run_mode)
850
851     all_rows = []
852     for alpha, nu in param_pairs:
853         solution = solve_burgers2d_fipy(alpha=alpha, nu=nu, cfg=cfg, force=False, verbose=True)
854
855         for n_data in dd_sizes:
856             print(f'\n[Data-driven] alpha={alpha}, nu={nu:g}, n_data={n_data}')
857             row, hist, _ = train_data_driven(solution, n_data=n_data, cfg=cfg,
858                                             seed=SEED + int(n_data) + int(1000*alpha))
859             append_result(row)
860             all_rows.append(row)
861
862         for n_data in pinn_sizes:
863             for opt_name in ['newton', 'adam']:
864                 print(f'\n[PINN] alpha={alpha}, nu={nu:g}, n_data={n_data}, opt={opt_name}')
865                 row, hist, _ = train_inverse_pinn(solution, n_data=n_data,
866                                                  pde_optimizer=opt_name, cfg=cfg,
867                                                  seed=SEED + int(n_data) + int(1000*alpha))
868                 append_result(row)
869                 all_rows.append(row)
870
871     return pd.DataFrame(all_rows)
872
873
874 if RUN_FULL_GRID:
875     if RESULTS_PATH.exists():
876         print('Ya existe results_df.csv. Se anadiran nuevas filas si corres de nuevo.')
877         results_df = run_full_experiment(CFG, RUN_MODE)
878     else:
879         print('RUN_FULL_GRID=False: no se lanza la grilla completa.')
880         print('Para generar Figuras 2--8 desde entrenamiento completo, pon RUN_FULL_GRID=True y
881             vuelve a ejecutar esta celda.')

```

```

882
883 #=====
884 # Celda 42
885 #=====
886 if RESULTS_PATH.exists():
887     results_df = pd.read_csv(RESULTS_PATH)
888 else:
889     print('No existe results_df.csv todavia. Usare solo la corrida individual como muestra
890           didactica.')
891     print('Para las Figuras 2--8 reales, ejecuta la grilla completa.')
892     results_df = pd.DataFrame([row_test])
893
894 results_df.head(), results_df.shape
895
896 #=====
897 # Celda 44
898 #=====
899 ALPHA_COLORS = {
900     0.5: '#1f77b4',
901     1.0: '#ff7f0e',
902     1.5: '#2ca02c',
903     2.0: '#d62728',
904 }
905
906 NU_MARKERS = {
907     1e-4: 'o',
908     1e-3: 'x',
909     1e-2: 's',
910 }
911
912
913 def _scatter_error(ax, df, y_col, title=None):
914     for _, row in df.iterrows():
915         alpha = float(row['alpha_true'])
916         nu = float(row['nu_true'])
917         ax.scatter(row['alpha_over_nu'], row[y_col],
918                   color=ALPHA_COLORS.get(alpha, 'black'),
919                   marker=NU_MARKERS.get(nu, 'o'),
920                   s=55, linewidths=1.8)
921     ax.axhline(0.1, color='black', linestyle='--', linewidth=1)
922     ax.set_xscale('log')
923     ax.set_yscale('log')
924     ax.set_xlabel(r'$\alpha/\nu$')
925     if title:
926         ax.set_title(title, fontsize=10)
927
928
929 def add_alpha_nu_legend(fig, anchor=(1.02, 0.5)):
930     alpha_handles = [Line2D([0], [0], marker='o', linestyle='None', color=c,
931                             label=f'{a:g}', markersize=7) for a, c in ALPHA_COLORS.items()]
932     nu_handles = [Line2D([0], [0], marker=m, linestyle='None', color='black',
933                           label=f'{nu:g}', markersize=7) for nu, m in NU_MARKERS.items()]
934     handles = [Line2D([], [], linestyle='None', label=r'$\alpha$')] + alpha_handles + \
935               [Line2D([], [], linestyle='None', label=r'$\nu$')] + nu_handles
936     fig.legend(handles=handles, loc='center left', bbox_to_anchor=anchor, frameon=True)
937
938
939 #=====
940 # Celda 46
941 #=====
942 def plot_figure2(results_df, save_path=None):
943     ncols = [2048, 8192, 32768]
944     y_rows = [
945         ('alpha_error', r'$\alpha$ error'),
946         ('nu_error', r'$\nu$ error'),
947         ('relative_error', 'Relative error')
948     ]
949

```

```

950     METHOD_COLORS = {
951         'PINN': 'tab:blue',
952         'Data-based fitting': 'tab:orange'
953     }
954
955     fig, axs = plt.subplots(3, 3, figsize=(8.2, 7.2), sharex=False)
956
957     for i, (y_col, y_label) in enumerate(y_rows):
958         for j, n in enumerate(ncols):
959             ax = axs[i, j]
960
961             d = results_df[
962                 (results_df['n_data'] == n) &
963                 (
964                     ((results_df['method'] == 'PINN') &
965                      (results_df['pde_optimizer'].astype(str).str.lower() == 'newton')) |
966                     (results_df['method'] == 'Data-based fitting')
967                 )
968             ].copy()
969
970             # Solo datos validos para escala logaritmica
971             d = d[
972                 (d['alpha_over_nu'] > 0) &
973                 (d[y_col] > 0) &
974                 np.isfinite(d['alpha_over_nu']) &
975                 np.isfinite(d[y_col])
976             ]
977
978             for method in ['PINN', 'Data-based fitting']:
979                 dm = d[d['method'] == method]
980
981                 if len(dm) == 0:
982                     continue
983
984                 ax.scatter(
985                     dm['alpha_over_nu'],
986                     dm[y_col],
987                     color=METHOD_COLORS[method],
988                     marker='o',
989                     s=35,
990                     alpha=0.9,
991                     edgecolors='none'
992                 )
993
994                 ax.axhline(0.1, color='black', linestyle='--', linewidth=1)
995
996                 # Fijar limites positivos aunque el panel quede vacio
997                 ax.set_xscale('log')
998                 ax.set_yscale('log')
999                 ax.set_xlim(5e1, 5e4)
1000                 ax.set_ylim(5e-3, 2)
1001
1002                 ax.set_title(f'# data = {n}', fontsize=9)
1003
1004                 if j == 0:
1005                     ax.set_ylabel(y_label)
1006
1007                 if i == 2:
1008                     ax.set_xlabel(r'$\alpha/\nu$')
1009
1010     method_handles = [
1011         Line2D([0], [0], marker='o', linestyle='None',
1012                color='tab:blue', label='PINN'),
1013         Line2D([0], [0], marker='o', linestyle='None',
1014                color='tab:orange', label='Data-based fitting')
1015     ]
1016
1017     fig.legend(
1018         handles=method_handles,

```

```

1019         title='Method',
1020         loc='lower center',
1021         ncol=2,
1022         frameon=True
1023     )
1024
1025     fig.suptitle('Figura 2 -- PINN Newton vs data-driven', y=1.02)
1026
1027     # Mas estable que tight_layout en este caso
1028     fig.subplots_adjust(
1029         left=0.08,
1030         right=0.98,
1031         top=0.90,
1032         bottom=0.16,
1033         wspace=0.30,
1034         hspace=0.45
1035     )
1036
1037     if save_path:
1038         fig.savefig(save_path, dpi=300, bbox_inches='tight')
1039
1040     plt.show()
1041
1042
1043 plot_figure2(results_df, CACHE_DIR / 'figure2.png')
1044
1045 #=====
1046 # Celda 48
1047 #=====
1048 def plot_figure3(results_df, save_path=None):
1049     ncols = [2048, 8192, 32768]
1050     y_rows = [('relative_error', 'Relative error'),
1051               ('alpha_error', r'$\alpha$ error'),
1052               ('nu_error', r'$\nu$ error')]
1053
1054     dbase = results_df[results_df['method'] == 'Data-based fitting']
1055     fig, axs = plt.subplots(3, 3, figsize=(8.4, 7.2))
1056     for i, (y_col, y_label) in enumerate(y_rows):
1057         for j, n in enumerate(ncols):
1058             ax = axs[i, j]
1059             d = dbase[dbase['n_data'] == n]
1060             _scatter_error(ax, d, y_col, title=f'# data = {n}')
1061             if j == 0:
1062                 ax.set_ylabel(y_label)
1063     add_alpha_nu_legend(fig, anchor=(1.02, 0.5))
1064     fig.suptitle('Figure 3 -- Data-driven estimation', y=1.02)
1065     fig.tight_layout()
1066     if save_path:
1067         fig.savefig(save_path, dpi=300, bbox_inches='tight')
1068     plt.show()
1069
1070 plot_figure3(results_df, CACHE_DIR / 'figure3.png')
1071
1072
1073 #=====
1074 # Celda 50
1075 #=====
1076 def plot_figure4(results_df, save_path=None):
1077     dbase = results_df[results_df['method'] == 'PINN']
1078     optimizers = ['newton', 'adam']
1079     ncols = [2048, 8192, 32768]
1080     y_groups = [('nu_error', r'$\nu$ error'), ('alpha_error', r'$\alpha$ error')]
1081
1082     fig, axs = plt.subplots(2, 6, figsize=(13.0, 5.2), sharex=False)
1083     for row_i, opt in enumerate(optimizers):
1084         for block_i, (y_col, y_label) in enumerate(y_groups):
1085             for j, n in enumerate(ncols):
1086                 ax = axs[row_i, block_i*3 + j]
1087                 d = dbase[(dbase['pde_optimizer'] == opt) & (dbase['n_data'] == n)]

```

```

1088         _scatter_error(ax, d, y_col, title=f'{opt}; # data = {n}')
1089         if block_i == 0 and j == 0:
1090             ax.set_ylabel(y_label)
1091         if block_i == 1 and j == 0:
1092             ax.set_ylabel(y_label)
1093     add_alpha_nu_legend(fig, anchor=(1.01, 0.5))
1094     fig.suptitle('Figure 4 -- PINN parameter estimation errors', y=1.02)
1095     fig.tight_layout()
1096     if save_path:
1097         fig.savefig(save_path, dpi=300, bbox_inches='tight')
1098     plt.show()
1099
1100 plot_figure4(results_df, CACHE_DIR / 'figure4.png')
1101
1102
1103 #####
1104 # Celda 52
1105 #####
1106 def plot_figure5(results_df, save_path=None):
1107     dbase = results_df[results_df['method'] == 'PINN']
1108     optimizers = ['newton', 'adam']
1109     ncols = [2048, 8192, 32768]
1110
1111     fig, axs = plt.subplots(2, 3, figsize=(8.0, 5.6))
1112     for i, opt in enumerate(optimizers):
1113         for j, n in enumerate(ncols):
1114             ax = axs[i, j]
1115             d = dbase[(dbase['pde_optimizer'] == opt) & (dbase['n_data'] == n)]
1116             _scatter_error(ax, d, 'relative_error', title=f'{opt}; # data = {n}')
1117             if j == 0:
1118                 ax.set_ylabel('Relative error')
1119     add_alpha_nu_legend(fig, anchor=(1.02, 0.5))
1120     fig.suptitle('Figure 5 -- PINN solution relative error', y=1.02)
1121     fig.tight_layout()
1122     if save_path:
1123         fig.savefig(save_path, dpi=300, bbox_inches='tight')
1124     plt.show()
1125
1126 plot_figure5(results_df, CACHE_DIR / 'figure5.png')
1127
1128
1129 #####
1130 # Celda 54
1131 #####
1132 def plot_data_driven_grid(results_df, y_col, ylabel, title, save_path=None):
1133     dbase = results_df[results_df['method'] == 'Data-based fitting']
1134     ncols = [128, 512, 2048, 8192, 32768, 131072]
1135     fig, axs = plt.subplots(2, 3, figsize=(8.0, 5.8))
1136     axs = axs.ravel()
1137     for ax, n in zip(axs, ncols):
1138         d = dbase[dbase['n_data'] == n]
1139         _scatter_error(ax, d, y_col, title=f'# data = {n}')
1140         ax.set_ylabel(ylabel)
1141     add_alpha_nu_legend(fig, anchor=(1.02, 0.5))
1142     fig.suptitle(title, y=1.02)
1143     fig.tight_layout()
1144     if save_path:
1145         fig.savefig(save_path, dpi=300, bbox_inches='tight')
1146     plt.show()
1147
1148 plot_data_driven_grid(results_df, 'relative_error', 'Relative error',
1149                       'Figure 6 -- Data-driven solution error', CACHE_DIR / 'figure6.png')
1150 plot_data_driven_grid(results_df, 'alpha_error', r'$\alpha$ error',
1151                       'Figure 7 -- Data-driven alpha error', CACHE_DIR / 'figure7.png')
1152 plot_data_driven_grid(results_df, 'nu_error', r'$\nu$ error',
1153                       'Figure 8 -- Data-driven nu error', CACHE_DIR / 'figure8.png')
1154
1155
1156 #####

```



```

1157 # Celda 56
1158 #=====
1159 def make_table2_from_results(results_df):
1160     d = results_df[results_df['method'] == 'PINN'].copy()
1161     if d.empty:
1162         return pd.DataFrame(columns=['PDE optimizer', '# data', 'alpha error', 'Relative error',
1163                                     'nu error'])
1164     tab = (
1165         d.groupby(['pde_optimizer', 'n_data'], as_index=False)
1166         .agg({'alpha_error': 'mean', 'relative_error': 'mean', 'nu_error': 'mean'})
1167         .rename(columns={
1168             'pde_optimizer': 'PDE optimizer',
1169             'n_data': '# data',
1170             'alpha_error': 'alpha error',
1171             'relative_error': 'Relative error',
1172             'nu_error': 'nu error'
1173         })
1174     )
1175     order = {'adam': 0, 'newton': 1}
1176     tab['_order'] = tab['PDE optimizer'].map(order)
1177     tab = tab.sort_values(['# data', '_order']).drop(columns='_order')
1178     return tab
1179
1180 table2_reproduced = make_table2_from_results(results_df)
1181 table2_reproduced
1182
1183 #=====
1184 # Celda 57
1185 #=====
1186 # Tabla 2 publicada en el paper, para comparar cuando ejecutes la grilla completa.
1187 table2_published = pd.DataFrame([
1188     ['Adam', 2048, 0.080, 0.152, 0.093],
1189     ['Newton', 2048, 0.086, 0.150, 0.056],
1190     ['Adam', 8192, 0.085, 0.139, 0.097],
1191     ['Newton', 8192, 0.085, 0.130, 0.065],
1192     ['Adam', 32768, 0.067, 0.139, 0.101],
1193     ['Newton', 32768, 0.088, 0.126, 0.071],
1194 ], columns=['PDE optimizer', '# data', 'alpha error', 'Relative error', 'nu error'])
1195
1196 table2_published
1197
1198 #=====
1199 # Celda 59
1200 #=====
1201
1202 def compare_table2(reproduced, published):
1203     if reproduced.empty:
1204         print('No hay resultados PINN entrenados suficientes todavia.')
1205         return None
1206     r = reproduced.copy()
1207     p = published.copy()
1208     r['PDE optimizer'] = r['PDE optimizer'].str.capitalize()
1209     merged = p.merge(r, on=['PDE optimizer', '# data'], suffixes=('_paper', '_run'))
1210     for col in ['alpha error', 'Relative error', 'nu error']:
1211         merged[col + ' diff'] = merged[col + '_run'] - merged[col + '_paper']
1212     return merged
1213
1214 comparison_table2 = compare_table2(table2_reproduced, table2_published)
1215 comparison_table2
1216
1217 #=====
1218 # Celda 61
1219 #=====
1220 # Guardado compacto para descargar resultados desde Colab.
1221 if RESULTS_PATH.exists():
1222     print('Resultados guardados en:', RESULTS_PATH)
1223 else:

```

```
1225     print('Todavía no existe archivo de resultados completo.')
```

```
1226
```

```
1227 print('Carpeta cache:', CACHE_DIR)
```

```
1228
```

```
1229
```

```
1230 #=====
```

```
1231 # Celda 62
```

```
1232 #=====
```

```
1233 !ls -lah /content
```

```
1234
```

```
1235 #=====
```

```
1236 # Celda 63
```

```
1237 #=====
```

```
1238 !zip -r /content/pinns_burgers2d_paper_cache.zip /content/pinns_burgers2d_paper_cache
```

```
1239
```

```
1240 #=====
```

```
1241 # Celda 64
```

```
1242 #=====
```

```
1243 from google.colab import files
```

```
1244 files.download("/content/pinns_burgers2d_paper_cache.zip")
```

Listing 13: Código completo del notebook Replica del paper.ipynb.

Referencias

- [1] New, A., Villafañe-Delgado, M., y Shugert, C. (2024). *Equation Identification for Fluid Flows via Physics-Informed Neural Networks*. Proceedings of the 41st International Conference on Machine Learning, PMLR 235. arXiv:2408.17271.
- [2] Raissi, M., Perdikaris, P., y Karniadakis, G. E. (2019). Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations. *Journal of Computational Physics*, 378, 686–707.
- [3] Guyer, J. E., Wheeler, D., y Warren, J. A. (2009). FiPy: Partial Differential Equations with Python. *Computing in Science & Engineering*, 11(3), 6–15.
- [4] Kingma, D. P. y Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980.
- [5] Baydin, A. G., Pearlmutter, B. A., Radul, A. A., y Siskind, J. M. (2017). Automatic Differentiation in Machine Learning: A Survey. *Journal of Machine Learning Research*, 18, 1–43.